

Key Vault(amm)

KeyVault-amm è un sistema **client-server** progettato per la gestione sicura e centralizzata delle credenziali, fondato sul paradigma **Zero Trust**. In questa architettura, la fiducia non è mai scontata, ma viene validata tramite verifiche crittografiche continue in ogni fase della comunicazione.

Il progetto, ideale per contesti **aziendali o domestici**, punta sulla facilità di deployment: il client automatizza i processi operativi dopo il setup iniziale, mentre il server richiede esclusivamente l'esecuzione all'interno di una rete privata accessibile.

Fase di Analisi

L'obiettivo principale di **KeyVault-amm** è fornire un ambiente protetto per la persistenza e la sincronizzazione di credenziali, minimizzando i rischi di intercettazione o accesso non autorizzato.

Requisiti Funzionali

I requisiti funzionali descrivono le capacità operative che il sistema deve offrire all'utente finale.

ID	Requisito	Descrizione
R_FUN_01	Configurazione Rete	Il sistema deve permettere all'utente di specificare l'indirizzo IP del server tramite interfaccia grafica per gestire connessioni remote o locali.
R_FUN_02	Importazione CA	Il sistema deve consentire l'importazione manuale del certificato <code>ca.crt</code> nello storage isolato del client per stabilire, al di fuori del canale di rete.
R_FUN_03	Registrazione (Enrollment)	Il client deve poter richiedere la firma di una CSR inviando un codice OTP valido, ottenendo così la propria identità digitale X.509.
R_FUN_04	Sblocco Multi-Fattore	Il sistema deve supportare lo sblocco della sessione

		crittografica sia tramite Master Password (derivazione PBKDF2) sia tramite chiave USB fisica.
R_FUN_05	Gestione Credenziali	L'utente deve poter inserire nuove credenziali e sincronizzare quelle esistenti, visualizzandole in chiaro solo dopo la decifratura locale.

Requisiti Non Funzionali

Questa sezione definisce i vincoli di sicurezza, prestazioni e architettura che il software deve rispettare.

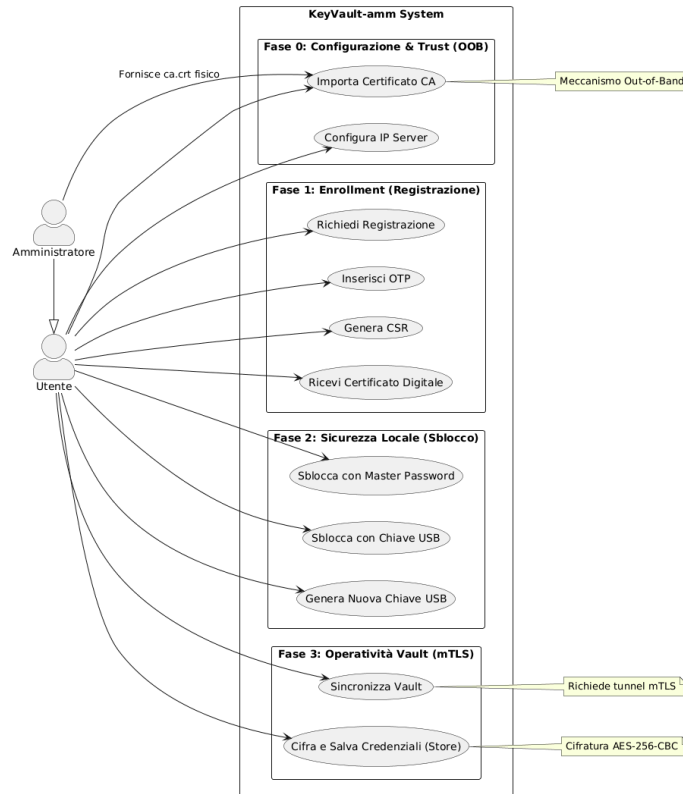
ID	Requisito	Descrizione
R_NFUN_01	Autenticazione mTLS	Ogni comunicazione deve essere protetta da TLS mutuo, garantendo l'identità certa sia del client che del server prima di ogni scambio dati.
R_NFUN_02	Cifratura at-Rest	Le password inviate al server devono essere cifrate con algoritmo AES-256-CBC.
R_NFUN_03	Isolamento Storage	Il sistema deve mantenere una netta separazione fisica tra le directory di storage del server e del client, impedendo accessi non autorizzati alle chiavi private.
R_NFUN_04	Gestione Concorrenza	Il server deve essere in grado di gestire sessioni multiple simultanee tramite un'architettura multithreading basata su POSIX Threads.

Use Case Diagram

Il diagramma dei casi d'uso illustra le funzionalità offerte dal sistema agli attori principali.

- **Utente:** L'utente finale che utilizza l'applicazione client.
È responsabile della configurazione del proprio ambiente, dell'importazione del **certificato** della CA e della gestione delle proprie credenziali tramite sblocco crittografico.

- **Amministratore:** Una specializzazione dell'utente che possiede privilegi di accesso al server. Gestisce l'infrastruttura PKI e distribuisce il certificato `ca.crt` tramite un canale sicuro esterno (Out-of-Band) per evitare attacchi di intercettazione sulla rete.



Activity Diagram

L'**Activity Diagram** rappresenta uno strumento fondamentale per modellare il comportamento dinamico del sistema. In questa sezione, i diagrammi vengono utilizzati per descrivere il **flusso** logico delle operazioni, evidenziando le sequenze di azioni, i punti di decisione e il coordinamento tra i diversi moduli funzionali di **KeyVault-amm**.

Activity Diagram: Bootstrap e Setup CA

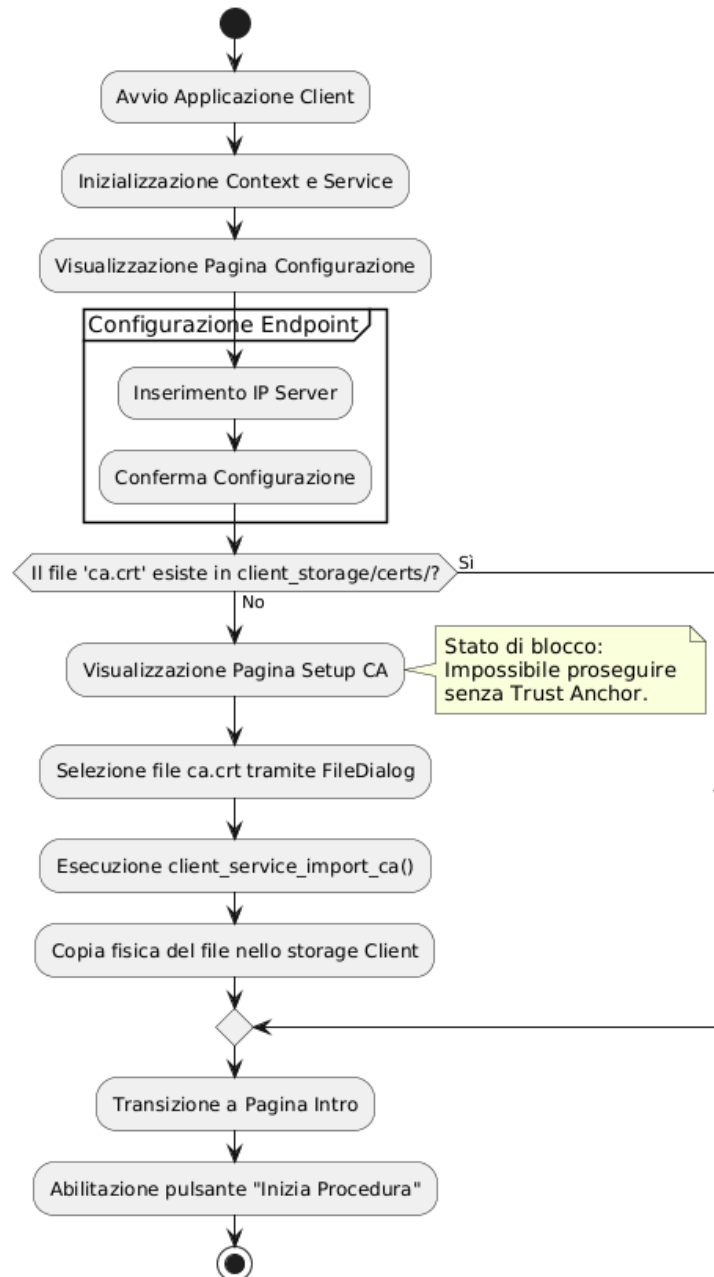
La procedura si articola in tre momenti chiave:

- L'utente definisce l'indirizzo IP del server, parametro che viene immediatamente memorizzato nel **Context Layer** per le sessioni future.
- Il sistema interroga lo storage (`client_storage/certs/`) per verificare la presenza del certificato radice della CA. Se il certificato è assente, l'interfaccia entra in uno stato di blocco logico che impedisce la navigazione verso le funzioni di rfiduciaegistrazione o vault.

- L'utente deve importare manualmente il file `ca.crt` ricevuto tramite un canale sicuro esterno.

Il Service si occupa di validare l'operazione e di salvare il file nel filesystem protetto, sbloccando così lo stato dell'applicazione e permettendo il passaggio alla fase di Enrollment.

Il certificato `ca.crt` è stato generato all'avvio dell'applicativo **server**, nella fase di creazione della Root Private CA.

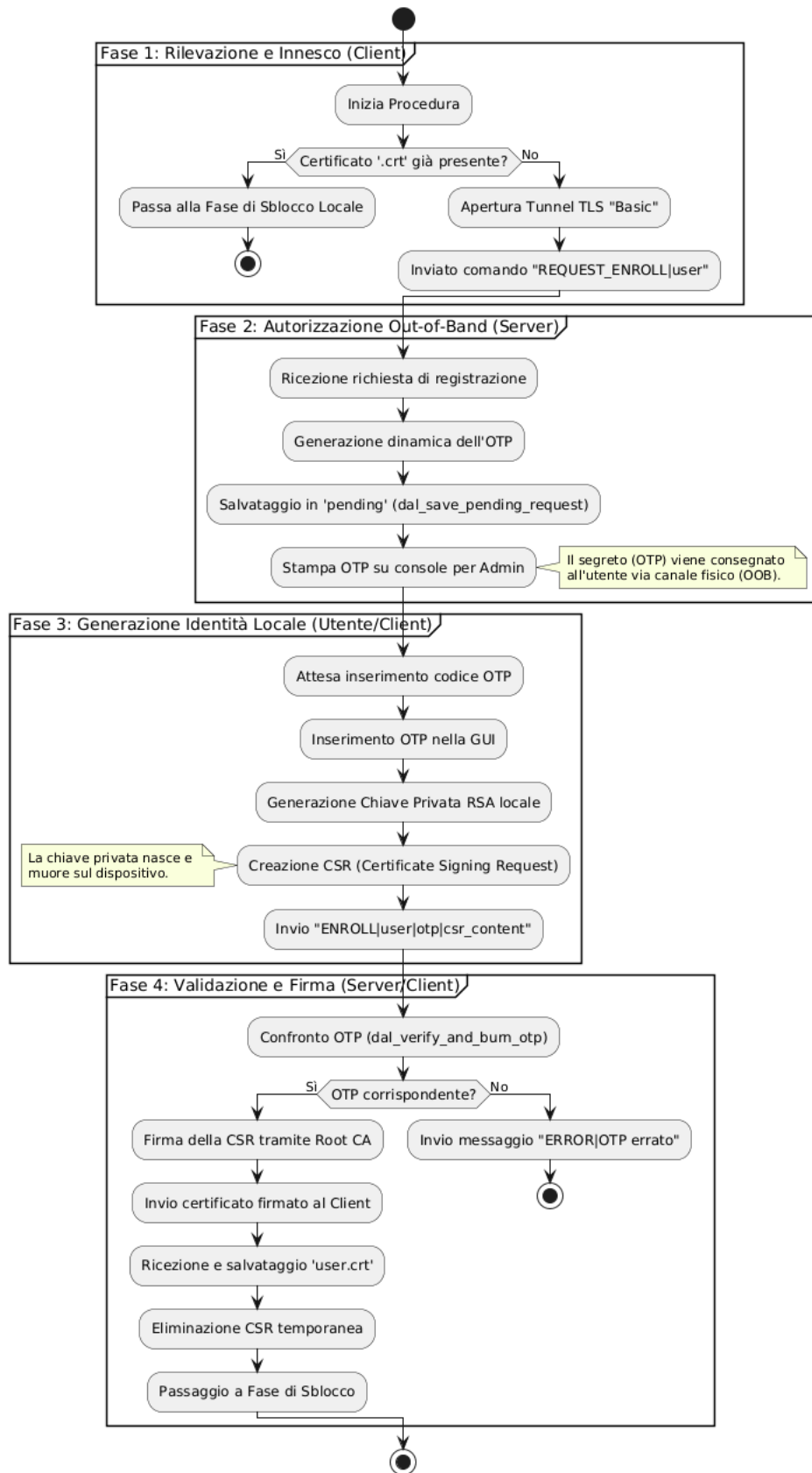


Questa fase rappresenta il punto di ingresso dell'applicazione e implementa il controllo di integrità sulla **Trust Anchor** del sistema. L'obiettivo primario è impedire che il client tenti qualsiasi comunicazione di rete senza aver prima stabilito una base di fiducia verificata, eliminando alla radice il rischio di attacchi *Man-in-the-Middle* durante il primo contatto.

Activity Diagram: Enrollment

La fase di **Enrollment** trasforma un client anonimo in un nodo autorizzato dell'architettura. Il cuore di questo processo è l'integrazione tra la logica e la **sicurezza fisica**, garantita dalla distribuzione **Out-of-Band (OOB)** delle credenziali iniziali, dell'**OTP**.

- Il client è proattivo; se non trova l'identità digitale nello storage, "bussa" al server per iniziare il processo di accreditamento.
- Il server genera il segreto (OTP) e lo affida all'amministratore per la consegna manuale. Questo rompe la catena di attacco digitale poiché il codice non viaggia sulla stessa rete dei dati.
- **Generazione Identità Locale (Client):** la chiave privata RSA viene generata sul client e non viene mai inviata al server; solo la richiesta di firma (CSR) lascia il dispositivo.
- **Validazione e Firma (Server):** Il cerchio si chiude. Il server brucia l'OTP (lo rende inutilizzabile per il futuro) e sigilla l'identità del client firmando il certificato.

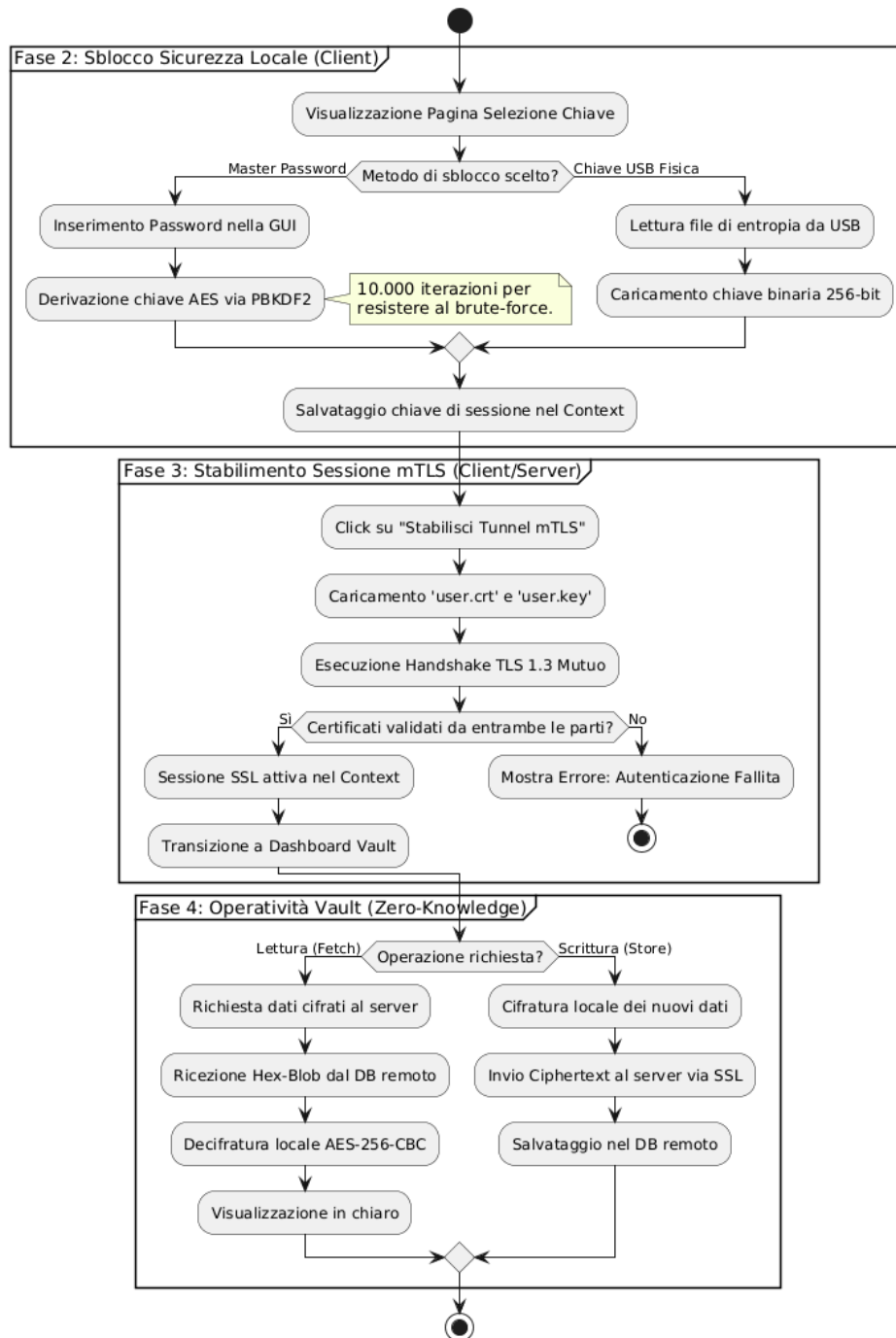


*Perché non il server non invia direttamente l'OTP tramite il canale **TLS base**?*

Anche se la richiesta parte digitalmente, la sicurezza "Zero Trust" viene mantenuta tramite un passaggio **Out-of-Band (OOB)** critico: una volta che il server genera l'OTP, questo deve essere comunicato all'utente tramite un canale esterno (es. console dell'amministratore, comunicazione verbale o documento fisico). Questo impedisce che un attaccante possa intercettare il codice e completare la registrazione al posto dell'utente legittimo.

Activity Diagram: Scelta Crittografica e Gestione Vault

Questa fase finale illustra il passaggio critico dalla protezione del dato "a riposo" (at-rest) alla sua gestione dinamica in rete.



- L'utente sceglie tra la conoscenza di una **Master Password**, elaborata tramite l'algoritmo **PBKDF2** per contrastare attacchi di forza bruta, o il possesso di una **chiave** privata contenuta in una **USB fisica** contenente l'entropia necessaria.
- Una volta derivata, la chiave viene salvata nella memoria RAM del **Client**, assicurando che non venga mai scritta su disco in chiaro.

- Il client avvia un tunnel **mTLS** dove entrambe le parti presentano i propri certificati X.509; il server accetta la connessione solo se riconosce il certificato rilasciato durante l'Enrollment.
- In fase di lettura o scrittura, la cifratura e decifratura **AES-256-CBC** avvengono sempre sul client; il server riceve e memorizza esclusivamente blob esadecimali cifrati, restando all'oscuro del contenuto reale delle password.

Fase di Progettazione

Stack Tecnologico

Durante il ciclo di vita del progetto, sono stati impiegati diversi strumenti per supportare le fasi di sviluppo, gestione del codice e progettazione.

- **Visual Studio Code**: ambiente di sviluppo IDE.
- **Git & GitHub**: utilizzati per il versioning del codice.
- **PlantUML** utilizzato per la creazione di diagrammi UML.

Tecnologie Software

L'architettura di **KeyVault-amm** è sviluppata interamente in **Linguaggio C**, una scelta dettata dalla necessità di avere un controllo granulare sulla gestione della memoria e sulle chiamate di sistema a basso livello, fondamentali in ambito crittografico.

Core & Security (OpenSSL Ecosystem)

Il nucleo di sicurezza del sistema si appoggia interamente alla libreria **OpenSSL**, lo standard industriale per la crittografia e le comunicazioni sicure.

- **Protocollo mTLS (TLS 1.3)**: utilizzato per garantire l'autenticazione mutua tra client e server.
- **Motore Crittografico Simmetrico**: il sistema sfrutta le primitive OpenSSL per l'implementazione dell'algoritmo **AES-256-CBC**. Le chiavi non vengono mai salvate direttamente, ma derivate tramite **PBKDF2** con 10.000 iterazioni di hashing per massimizzare la resistenza ad attacchi brute-force.
- **Gestione PKI e RSA**: La logica di generazione delle chiavi e delle **CSR (Certificate Signing Request)** è gestita internamente tramite algoritmi RSA a 2048 bit, garantendo l'integrità dell'identità digitale dell'utente.

Front-End (GTK 4 & Adwaita)

L'interfaccia utente del client è realizzata come un'applicazione desktop nativa per sistemi Linux-based.

- **Toolkit Grafico GTK 4:** Scelta per le elevate prestazioni e la gestione del rendering tramite nodi grafici. Permette una gestione fluida degli stati dell'interfaccia (configurazione, enrollment, vault).
- **Architettura Event-Driven:** L'integrazione con il **GMainLoop** di GLib permette di gestire in modo asincrono le risposte del server e gli input dell'utente, evitando il blocco della UI durante le operazioni crittografiche pesanti.

Back-End & Concorrenza (POSIX Threads)

Il server è progettato per essere leggero e scalabile, operando come un servizio di rete multithreaded.

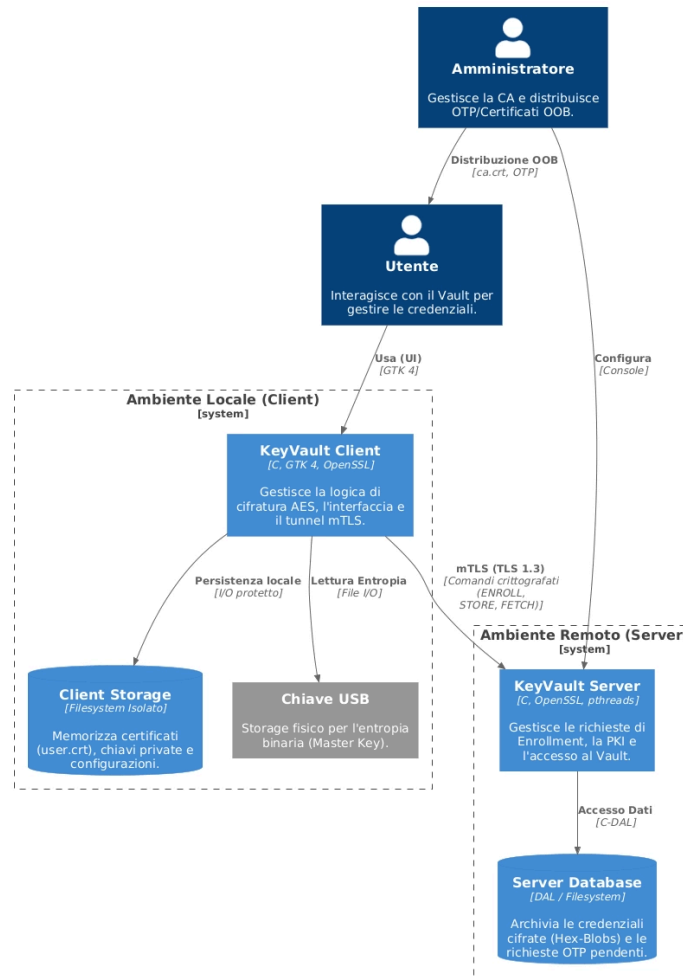
- **Gestione Threading (pthreads):** Ogni nuova connessione client viene gestita da un thread POSIX dedicato, permettendo al server di servire più utenti simultaneamente senza colli di bottiglia.

Architettura e Design del Sistema

Vista ad Alto Livello (Context Diagram)

Il **Context Diagram** rappresenta il livello più alto di astrazione dell'architettura.

Il suo scopo è mostrare il sistema e identificare le interazioni con gli attori e i supporti esterni.



Per garantire la massima sicurezza, l'infrastruttura è stata suddivisa in tre domini principali:

- **KeyVault Client:** funge da unico punto di accesso per l'utente.
- **KeyVault Server (Remoto):** Un servizio **multithreaded** che implementa la logica di una **Certification Authority (CA)** privata. Il server non ha mai accesso alle chiavi di cifratura dell'utente; il suo compito è validare l'identità (mTLS) e fungere da deposito per i dati già cifrati.
- **Canali di Comunicazione:**
 - **mTLS (Mutual TLS):** Il protocollo primario basato su **TLS 1.3**, dove sia il client che il server devono esibire un certificato X.509 valido per comunicare.
 - **Canale OOB (Out-of-Band):** Un canale fisico/esterno utilizzato per la consegna manuale del certificato della CA e dell'OTP di registrazione.

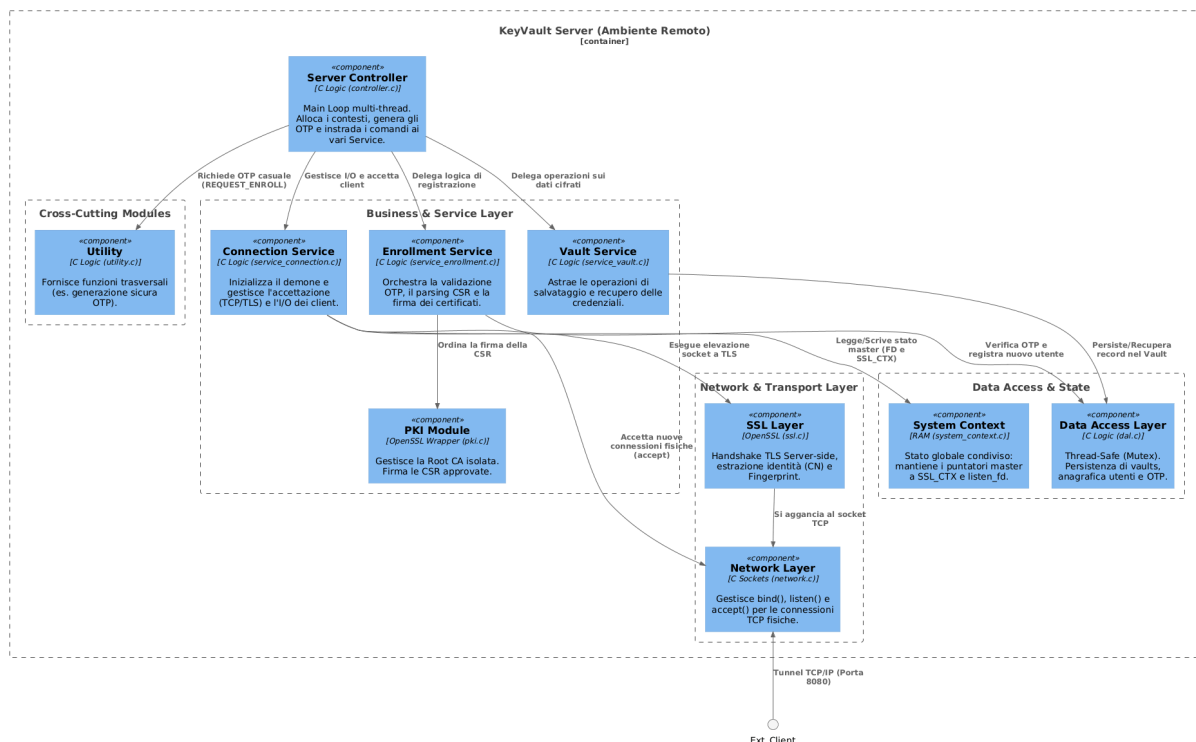
Architettura del Sistema

L'infrastruttura di KeyVault è fondata sulla **Separazione delle Responsabilità (SoC)**, un principio cardine che permette di isolare la logica decisionale dalla gestione dei dati e dalle comunicazioni a basso livello. Il sistema si presenta come un ecosistema diviso in due entità simmetriche, Client e Server, che condividono una filosofia di design modulare.

Nodo Server: Architettura Tiered (Controller-Service-DAL)

Il server è stato progettato per garantire scalabilità e sicurezza attraverso una struttura a tre strati:

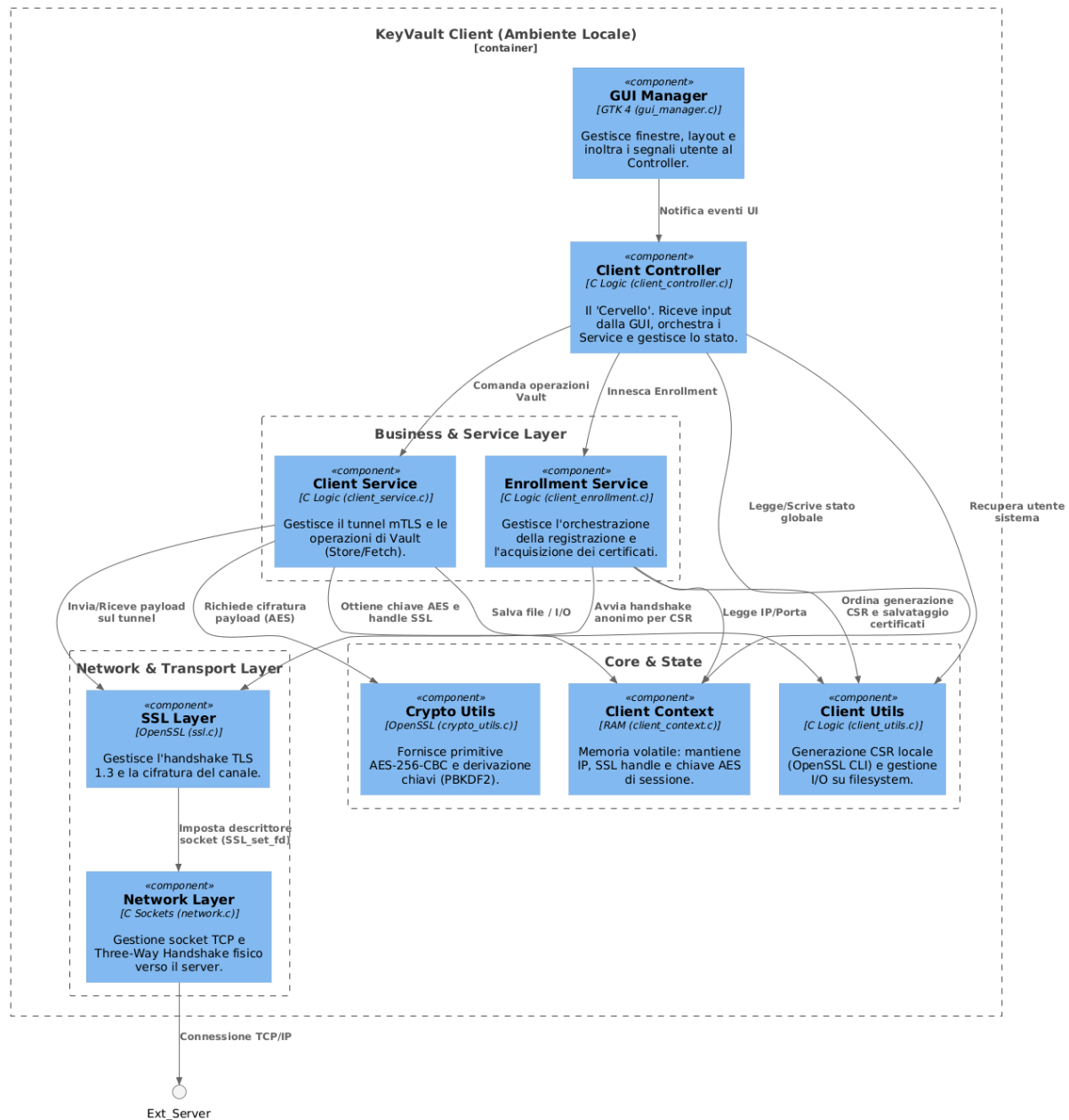
- **Controller Layer:** Funge da *Entry-point* unificato. Il modulo `controller.c` gestisce il loop di ascolto e istanzia thread indipendenti per ogni client, garantendo l'isolamento delle risorse tramite la struttura `ClientContext`.
- **Service Layer:** Rappresenta l'orchestratore della logica di business. I moduli `service_connection`, `service_enrollment` e `service_vault` coordinano le operazioni complesse senza mai accedere direttamente allo storage, interfacciandosi con la PKI e le utility di sistema.
- **Data Access Layer (DAL):** Gestisce la persistenza atomica sul filesystem. Grazie all'uso di **Mutex**, il modulo `dal.c` garantisce l'integrità dei dati e previene *race condition* durante l'accesso simultaneo ai vault o ai registri OTP.



Nodo Client: Controller-Service e Context

Il client adotta una struttura efficace per la gestione della sessione locale:

- **Controller Layer:** Riceve gli input dalla GUI (GTK 4) e pilota lo stato dell'applicazione. Agisce come mediatore tra l'utente e i servizi di backend.
- **Service Layer:** Gestisce la logica di crittografia locale (AES-256) e l'invio delle richieste al server. Include moduli specifici per l'acquisizione del certificato mTLS e per la comunicazione sicura.
- **Client Context:** È il componente critico per la gestione dello stato. In esso sono memorizzati i dati "core" della sessione, come l'indirizzo IP del server, il descrittore del tunnel SSL attivo, lo username dell'utente loggato e la chiave crittografica derivata. Questo modulo permette ai diversi servizi di operare in modo coordinato senza dover passare costantemente parametri globali, fungendo da "singola fonte di verità" per l'intera durata del processo.

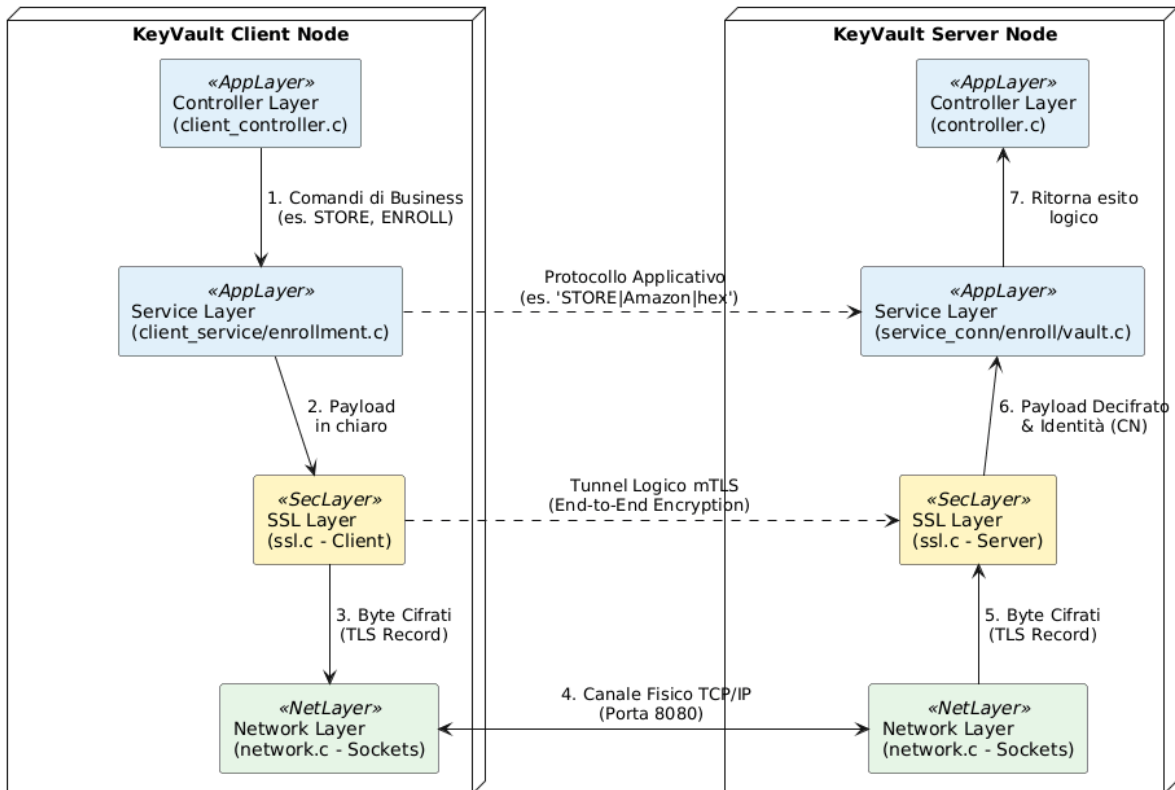


Stack di Comunicazione Simmetrico (SSL-Network)

Entrambi i nodi sono dotati di uno stack di rete identico, suddiviso in due layer:

1. **SSL/TLS Layer:** Implementato tramite la libreria OpenSSL, questo strato incapsula i dati in un tunnel crittografato mTLS. Si occupa dell'handshake, della verifica dei certificati e della cifratura/decifratura end-to-end.
2. **Network Layer:** Basato su socket BSD standard (POSIX), gestisce la creazione dei descrittori di file (file descriptors), il binding delle porte e il trasporto fisico dei pacchetti TCP.

L'isolamento di questi due strati permette alla logica applicativa (Controller e Service) di operare su stream di dati "trasparenti", delegando interamente la sicurezza e la connettività ai layer sottostanti.



Fase di Implementazione

In questo capitolo viene descritta la realizzazione pratica del sistema **KeyVault-amm**, analizzando l'organizzazione del codice sorgente e le scelte tecniche effettuate durante lo sviluppo dei singoli moduli.

Package Diagram

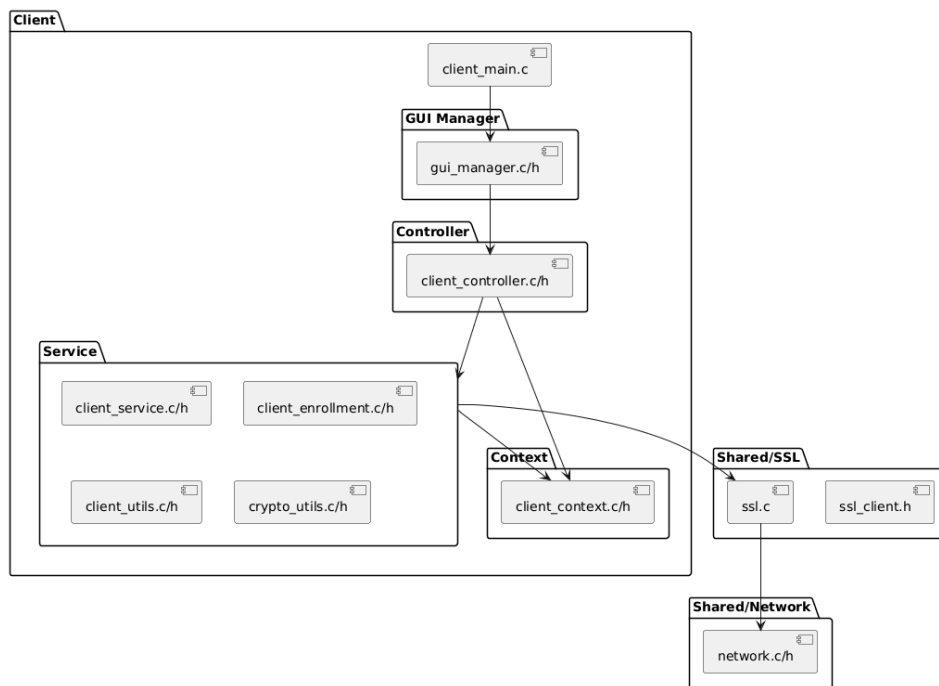
La struttura del progetto è stata suddivisa in pacchetti logici per facilitare la manutenzione e il disaccoppiamento tra le componenti. Di seguito viene illustrata l'organizzazione delle directory per i due nodi principali e per i moduli condivisi.

Struttura Client

Il client è organizzato per supportare l'interfaccia grafica e la gestione della sessione locale:

- **Context:** Gestisce i dati "core" della sessione corrente (`client_context.c`).

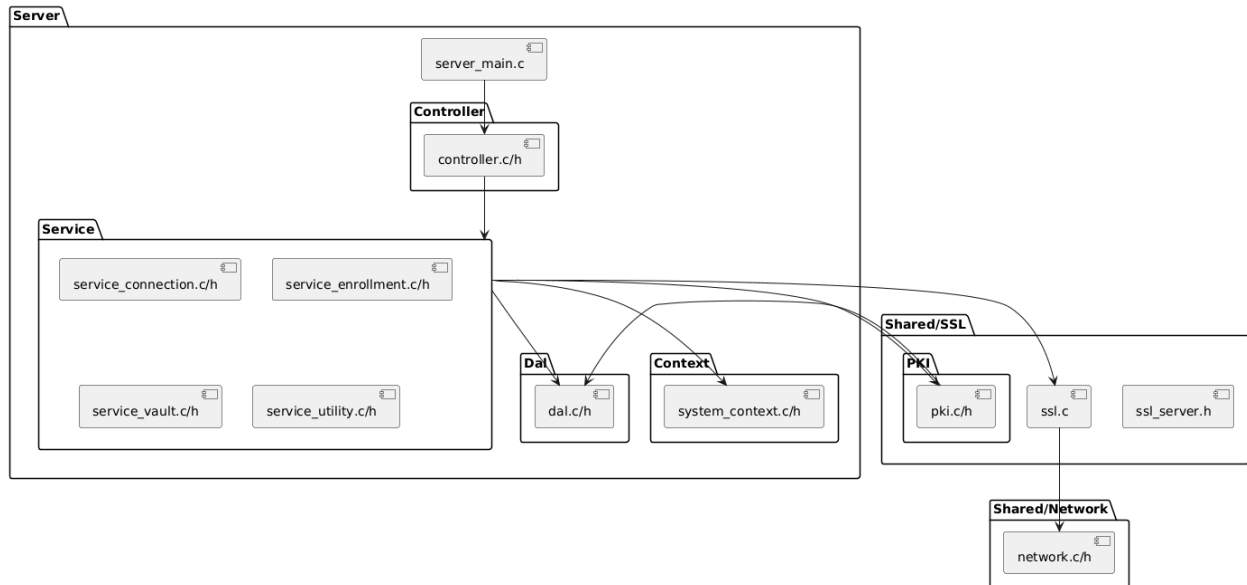
- **Controller:** Contiene la logica di controllo dell'applicazione (`client_controller.c`).
- **GUI Manager:** Include i file relativi all'interfaccia GTK 4 e i relativi asset visivi (`gui_manager.c`).
- **Service:** Raccoglie i servizi di crittografia (`crypto_utils.c`), gestione dell'enrollment (`client_enrollment.c`) e comunicazione applicativa (`client_service.c`).



Struttura Server

L'organizzazione del server è orientata alla gestione multi-thread e alla separazione dei servizi di business:

- **Context:** Contiene la gestione dello stato globale del sistema (`system_context.c`).
- **Controller:** Ospita l'entry-point multithread del server (`controller.c`).
- **Dal:** Modulo dedicato esclusivamente all'accesso ai dati e alla persistenza sul filesystem (`dal.c`).
- **Service:** Suddivide la logica di business in componenti verticali: `service_connection.c` , `service_enrollment.c` , `service_vault.c` e le utilità comuni in `service_utility.c` .



Moduli Condivisi

Per garantire la simmetria del protocollo di comunicazione, alcuni moduli sono condivisi o strutturalmente identici tra i due nodi:

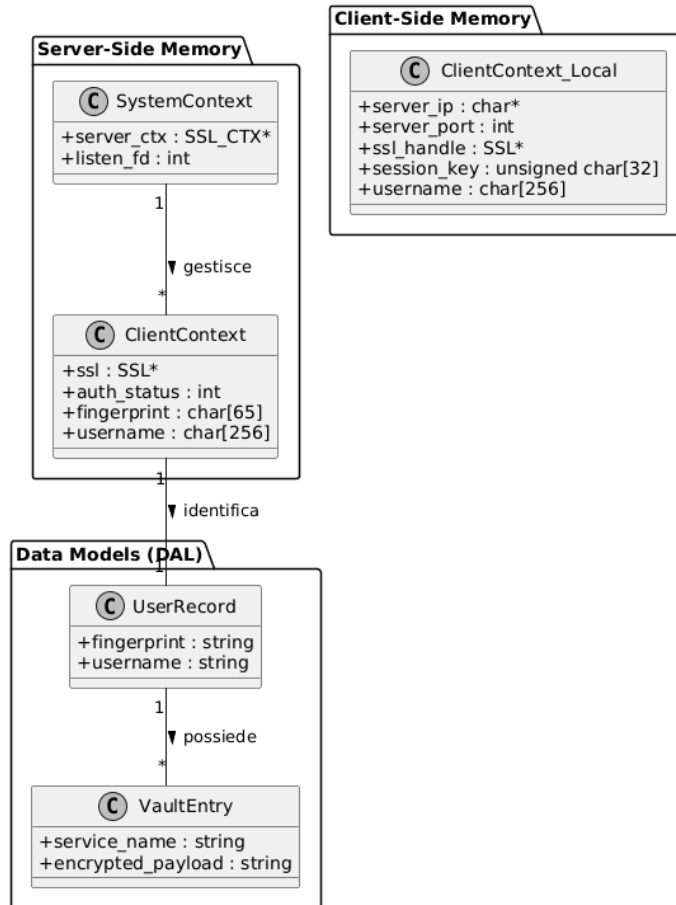
- **Network:** Gestisce le primitive dei socket TCP (`network.c`).
- **SSL:** Include la logica di gestione del tunnel TLS (`ssl.c`) e la libreria PKI interna del server (`pki.c`).

Modellazione dei Dati e Persistenza

La gestione dei dati in **KeyVault-amm** è stata progettata per garantire l'integrità delle informazioni sia in memoria volatile (RAM) che durante la memorizzazione su disco. Il sistema distingue nettamente tra lo **Stato di Sessione**, gestito tramite contesti dinamici, e la **Persistenza dei Dati**, affidata a un layer di accesso ai dati (DAL) thread-safe.

Strutture Dati - Class Diagram

Per visualizzare l'organizzazione logica delle informazioni, il seguente **Class Diagram** illustra le strutture (`struct`) utilizzate per gestire la sessione e la rappresentazione dei record nel database.



- **ClientContext (Server):** Questa struttura è il pilastro del multi-threading lato server. Viene allocata dinamicamente tramite `calloc` all'interno del loop di accettazione del controller per garantire che ogni thread lavori su un'area di memoria isolata. Contiene il riferimento all'oggetto `SSL*` per la comunicazione cifrata e i metadati di identità del client (`fingerprint` e `username`) estratti durante l'handshake mTLS.
- **SystemContext (Server):** Mantiene lo stato globale del demone server, inclusi il descrittore del socket in ascolto (`listen_fd`) e il contesto OpenSSL master (`SSL_CTX`), necessari per inizializzare le nuove connessioni.
- **ClientContext (Client):** Lato client, questa struttura funge da "Single Source of Truth", memorizzando temporaneamente la chiave AES di sessione derivata e i parametri di connessione necessari per il corretto funzionamento dei servizi.

Il **Data Access Layer (DAL)** astrae la complessità del filesystem, fornendo interfacce semplici per il salvataggio e il recupero delle informazioni.

- **Thread-Safety:** Poiché più thread possono tentare di registrare nuovi utenti o validare OTP contemporaneamente, il modulo `dal.c` implementa un Mutex statico (`dal_mutex`). Questo meccanismo di sincronizzazione garantisce che le operazioni sui file critici

come `users.dat` e `pending_requests.dat` siano atomiche, prevenendo la corruzione dei dati causata da *race conditions*.

- **Organizzazione del Database:**

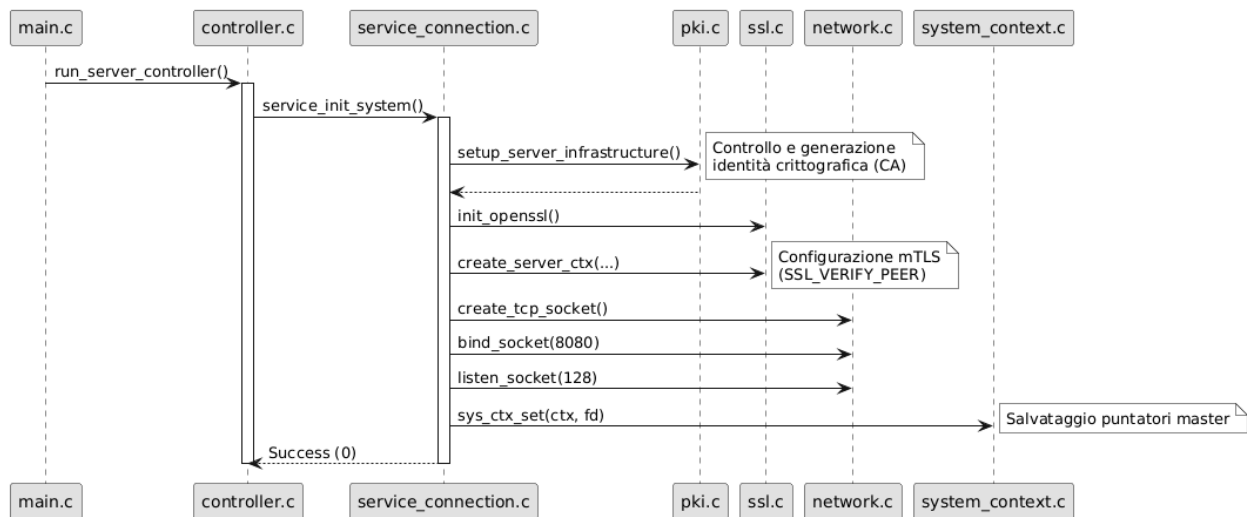
- **Anagrafica:** Il file `users.dat` mappa i fingerprint crittografici agli username.
- **Vault Individuali:** Le credenziali sono memorizzate in file binari separati (`[fingerprint].dat`) all'interno della directory `vaults/`. Ogni record è salvato in formato append-only per minimizzare il rischio di perdita dati, seguendo lo schema `Servizio|Blob_Cifrato`.

Inizializzazione e Bootstrapping del Server

L'avvio del server non è una semplice esecuzione di istruzioni, ma un processo di **bootstrapping**. Prima di poter accettare una connessione, il sistema deve garantire che l'infrastruttura crittografica sia integra e che il socket di rete sia configurato per l'ascolto passivo.

Sequence Diagram: Boot Sistema

Questo diagramma isola esclusivamente la preparazione dei componenti core prima dell'apertura del socket di ascolto.



▼ Focus `setup_server_infrastructure`

Il server agisce come una **Certification Authority (CA) locale**:

- **Verifica Idempotente:** Tramite `access()`, il server controlla la presenza di `ca.crt`. Se il file esiste, il boot prosegue riutilizzando la catena di fiducia esistente, permettendo ai client già registrati di collegarsi senza interruzioni.

- **Generazione Root CA:** In caso di prima esecuzione, viene generata una chiave RSA a **4096 bit** e un **certificato self-signed** valido per un anno.

```
// 1. Generazione Root CA (Certificato + Chiave Privata)
system("openssl req -x509 -newkey rsa:4096 "
      "-keyout " SERVER_CERTS_PATH "ca.key "
      "-out " SERVER_CERTS_PATH "ca.crt "
      "-days 365 -nodes -subj '/CN=KeyVault Root CA' 2>/dev/null");
/*
    -x509: indica che stiamo creando un certificato per una Root CA,
           non una semplice richiesta
    -newkey rsa:4096, genera una chiave RSA a 4096 bit
    -nodes: la chiave privata ca.key non è protetta da password,
           per cui un attaccante accede al filesystem e potrebbe
           firmare qualsiasi certificato a nome della nostra CA
    -subj: imposta il Common Name a KeyVault Root CA
*/
```

- **Certificato del Nodo Server:** Il server genera una propria chiave privata (2048 bit) e una **CSR** (Certificate Signing Request), che firma immediatamente con la chiave privata della Root CA da esso gestito. Questo garantisce che anche il server sia un'identità "certificata" all'interno del proprio ecosistema.

```
// 2. Generazione Chiave RSA a 2048 bit Privata del Server
system("openssl genrsa -out " SERVER_CERTS_PATH "server.key 2048 2>/dev/null");

// 3. Generazione CSR per il Server, passando nel
system("openssl req -new -key " SERVER_CERTS_PATH "server.key "
      "-out " SERVER_CERTS_PATH "server.csr -subj '/CN=KeyVault Server' 2>/dev/null");
```

Ma come si arriva a **Generare una CSR**?

La chiave privata `server.key` generata, non è un semplice valore ma una **struttura dati** complessa definita dallo standard **PKCS#1**.

Essa contiene **parametri matematici** — tra cui il modulo `n`, l'esponente pubblico `e`, l'esponente privato `d` e i fattori primi `p` e `q` — necessari per le **operazioni** di **decifrazione** e **firma**.

Da questa struttura vengono estratti esclusivamente il **modulo** `n` e l'**esponente pubblico** `e` per derivare la **chiave pubblica**, la quale viene inserita nel modulo della **CSR** (Certificate Signing Request).

Questo processo segue lo standard **PKCS#10**, specifico per la richiesta di certificati X.509.

```
CertificationRequestInfo ::= SEQUENCE {
    version          INTEGER { v1(0) } (v1,...),
    subject          Name,
    subjectPKInfo    SubjectPublicKeyInfo{{ PKInfoAlgorithms }},
    attributes       [0] Attributes{{ CRIAttributes }}
}

SubjectPublicKeyInfo { ALGORITHM : IOSet} ::= SEQUENCE {
    algorithm        AlgorithmIdentifier {{IOSet}},
    subjectPublicKey  BIT STRING
}

CertificationRequest ::= SEQUENCE {
    certificationRequestInfo CertificationRequestInfo,
    signatureAlgorithm  AlgorithmIdentifier{{ SignatureAlgorithms }},
    signature            BIT STRING
}
```

- **CertificationRequestInfo** : questo è il blocco principale che contiene il *Common Name* (es. KeyVault Server), la chiave pubblica (`n,e`) e altri attributi opzionali. È l'unica parte su cui viene calcolato l'hash.
- **SignatureAlgorithm** : Un identificatore che specifica quale algoritmo è stato usato per firmare (es. `sha256WithRSAEncryption`).
- **Signature** : risultato della cifratura dell'hash dei dati con la chiave privata.

Quindi, la firma è parte integrante dell'oggetto CSR.

```
// 4. Firma del certificato Server tramite la propria CA
system("openssl x509 -req -in " SERVER_CERTS_PATH "server.csr "
      "-CA " SERVER_CERTS_PATH "ca.crt -CAkey " SERVER_CERTS_PATH
```

```

"ca.key "
    "-CAcreateserial -out " SERVER_CERTS_PATH "server.crt -days 3
65 2>/dev/null");
/*
    x509 utility per la gestione certificati nello standard X.50
9.
    -req comunica a OpenSSL che il file in INPUT è una CSR
    -in server.csr specifica il file di input (CSR effettiva)
    -CA ca.crt indica il certificato della ROOT CA

    -CAkey ca.key indica la chiave privata della key privata dell
a CA
    Solo chi possiede questa chiave può emettere certificati valid
i per l
    'infrastruttura

    -CAcreateserial: ordina a OpenSSL di creare un file che ti
ene
    traccia del numero di serie dei certificati,
    così che ogni certificato abbia un numero univoco
    -out server.crt, definisce il file di output che verrà usa
to dal server      per presentarsi ai client durante l'handshake
TLS
    -days 365: data scadenza certificato

*/

```

La PKI sa esattamente in quale byte della struttura PKCS#10 trovarla.

1. Estrae **CRI** e ne ricalcola l'**hash** in locale.
2. Estrae la **Signature** e la **decifra** usando la **chiave pubblica** **(e,n)** presente all'interno della **CRI**.
3. **Confronta i due hash** e se corrispondono, la PKI ha la prova che il **pacchetto** è **integro** e **che chi lo ha inviato possiede la chiave privata associata**.

Ciò impedisce ad un malintenzionato a richiedere un **certificato** per una **chiave pubblica** di cui non detiene il controllo.

4. La **CA** estrae il **CN** e la **chiave pubblica** dalla **CSR** e li inserisce nella struttura del nuovo **certificato** **server.crt**.

5. La **CA** prende l'**hash** del **certificato** e lo **cifra** con la sua **ca.key** **privata**, allegata al **file**.

Il file **server.crt** ora contiene l'identità del server, la chiave pubblica del server e la **firma della CA** che garantisce l'autenticità di tutto il pacchetto.

▼ Focus `create_server_ctx`

In questa fase si definisce il protocollo di sicurezza applicativo:

```
SSL_CTX *create_server_ctx(const char *cert_file, const char *key_
file, const char *ca_file) {
    SSL_CTX *ctx = SSL_CTX_new(TLS_server_method());
    if (!ctx) return NULL;
    if (SSL_CTX_use_certificate_file(ctx, cert_file, SSL_FILETYPE_PEM) <= 0 ||
        SSL_CTX_use_PrivateKey_file(ctx, key_file, SSL_FILETYPE_PEM) <= 0) {
        SSL_CTX_free(ctx); return NULL;
    }
    if (!SSL_CTX_check_private_key(ctx)) {
        SSL_CTX_free(ctx);
        return NULL;
    }
    if (SSL_CTX_load_verify_locations(ctx, ca_file, NULL) <= 0) {
        SSL_CTX_free(ctx);
        return NULL;
    }
    SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);
    return ctx;
}
```

- **SSL_CTX** è il contesto che funge da fabbrica per tutte le future connessioni ai client, inizializzandolo con `TLS_server_method()`.
- Vengono caricati il **certificato** del **server** e la relativa **chiave privata** e il certificato della CA.
- **Attivazione mTLS**: La funzione `SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL)` impone il **Mutual TLS**. Senza questa riga, il server non richiederebbe il certificato al client, vanificando l'intera architettura basata su identità crittografica.

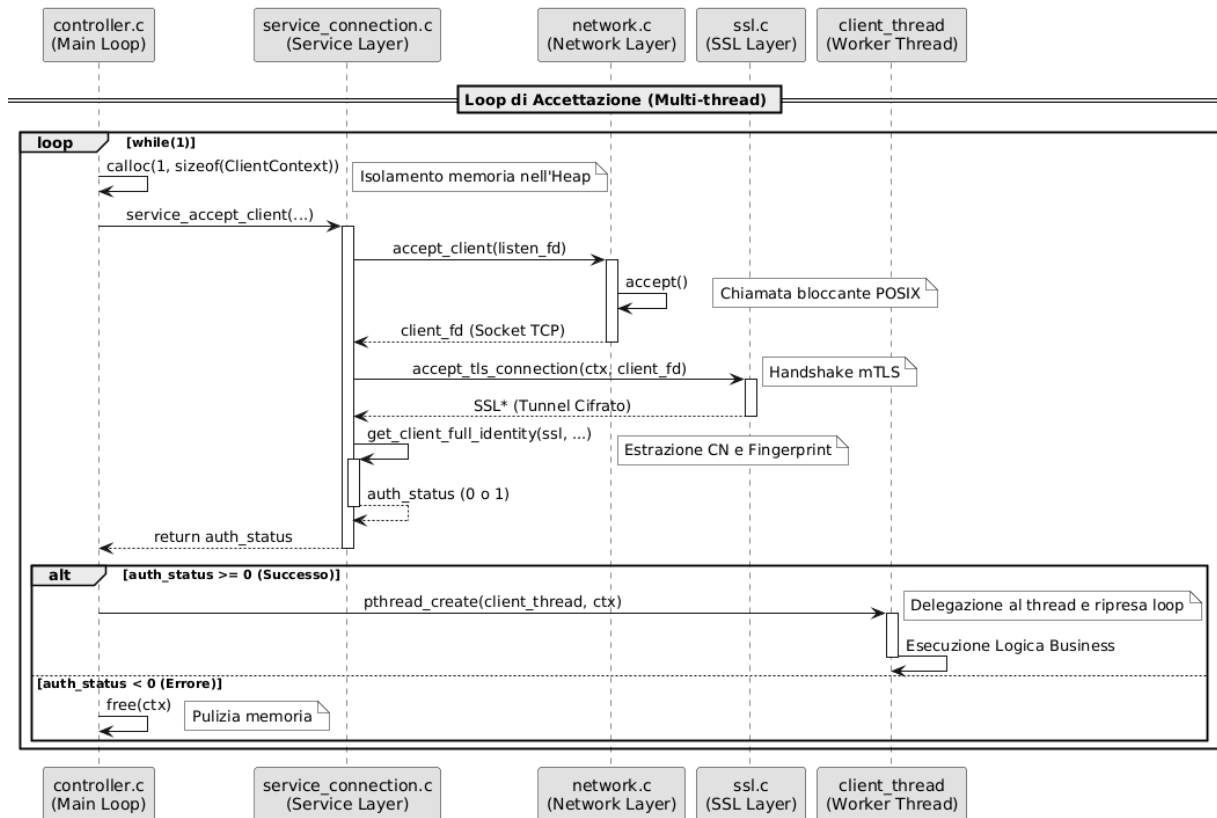
Successivamente c'è la fase di creazione della **socket** su cui il server sarà in ascolto, **bindata** sulla porta 8080 e messa in ascolto (con un **backlog** in input eventualmente, ma

nei test sfrutteremo il limite di **default 5**).

Infine, il server salva il proprio **contesto** nel proprio **core**.

Sequence Diagram: Loop di accettazione

Una volta completato il boot, il server entra nel loop infinito del `controller.c`. Questa fase implementa un modello di gestione della concorrenza **Multi-threaded**, dove ogni client viene isolato in un proprio contesto di memoria.



1. **Allocazione Preventiva ed Isolata:** L'uso di `calloc` all'inizio del loop garantisce che ogni potenziale client abbia già un `ClientContext` pronto ad accogliere i suoi dati identificativi (`SSL handle`, `username`, `fingerprint`). Se la connessione fallisce, la memoria viene liberata immediatamente tramite `free(ctx)`.

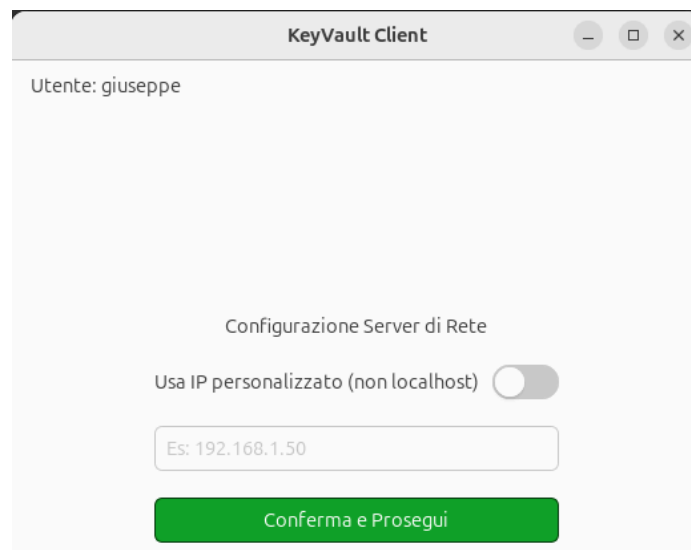
- **Controller:** Gestisce esclusivamente il flusso (loop, allocazione, creazione thread).
- **Service:** Gestisce l'orchestrazione della connessione, decidendo se il **client** è in fase di **enrollment** (anonimo) o **autenticato** tramite **certificato**.

2. **Identificazione tramite mTLS:** La funzione `get_client_full_identity` è il punto in cui il server tenta di conoscere il suo interlocutore.

- Estrae il **Common Name (CN)** per scopi di visualizzazione.
 - Estrae il **Fingerprint** (hash del certificato), che diventerà la chiave univoca per il modulo DAL nel recupero del vault corretto.
3. **Concorrenza Reattiva:** Utilizzando `pthread_create` dopo che `service_accept_client` ha restituito il controllo, il thread principale (Main Loop) rimane bloccato solo per il tempo strettamente necessario all'handshake. Una volta creato il thread figlio, il controller torna istantaneamente in `accept()`, rendendo il server capace di gestire più tentativi di connessione quasi simultaneamente.

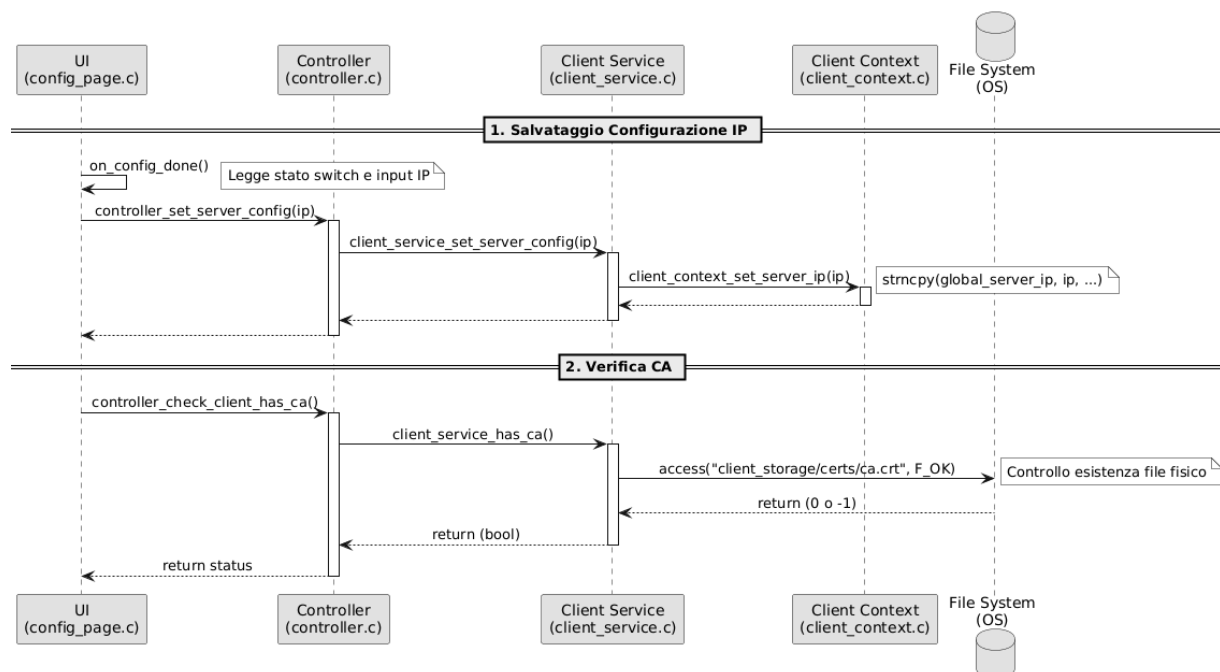
Configurazione del Client

Al primo avvio, l'applicazione KeyVault presenta un'interfaccia di configurazione di rete. Questa fase è fondamentale per stabilire l'**endpoint del server** con cui il client dovrà negoziare l'handshake TLS e l'eventuale processo di enrollment (se necessario).



Sequence Diagram: Configurazione Endpoint

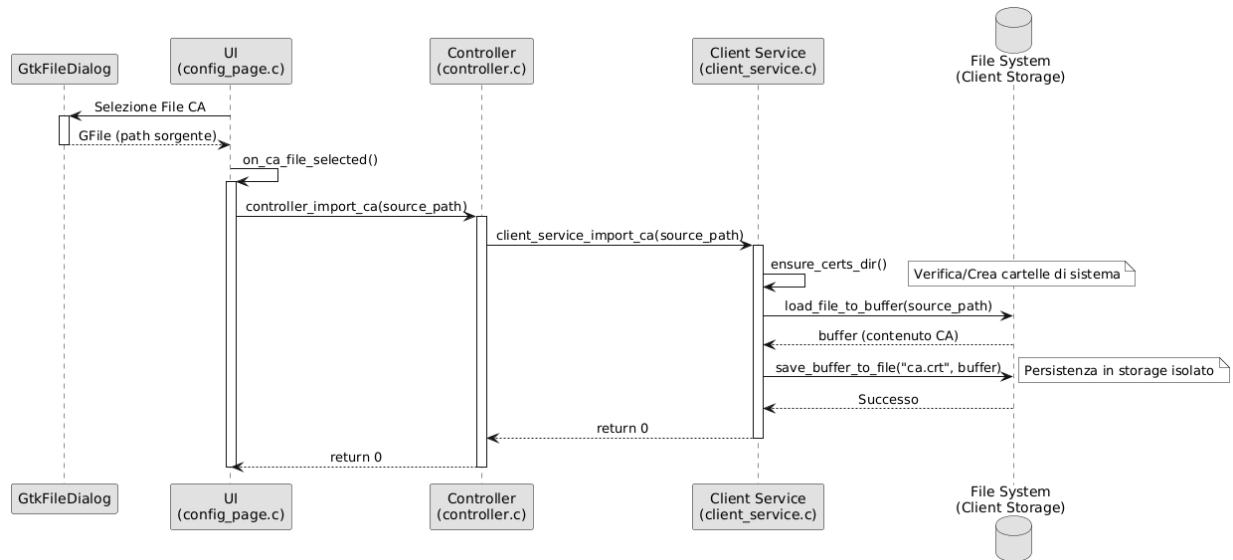
L'utente può scegliere se connettersi a un'istanza locale (*localhost*) o specificare un indirizzo IP personalizzato. Al clic sul pulsante **"Conferma e Prosegui"**, l'applicazione non si limita a salvare l'indirizzo, ma esegue un **controllo** preventivo sullo stato crittografico del client, verificando se è in possesso del proprio **certificato**.



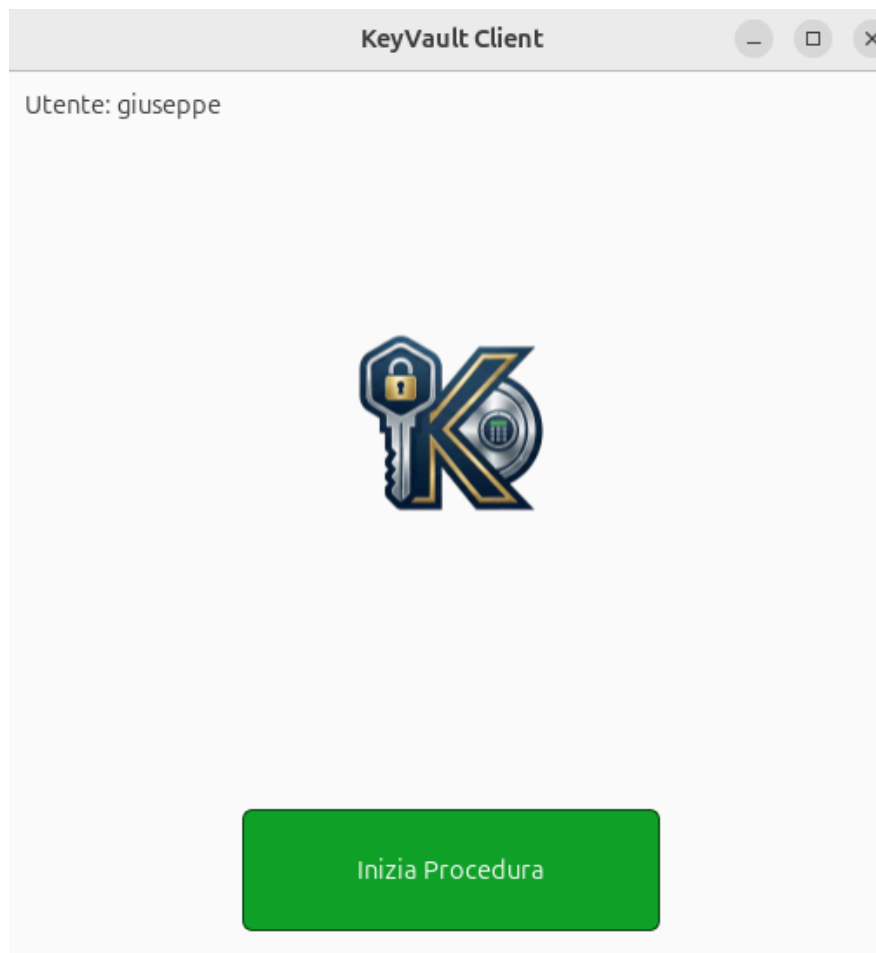
Sequence Diagram: Import Certification Authority

In questa fase, il client interagisce con il file system locale per selezionare il certificato `ca.crt` precedentemente ottenuto dal server **OOB**.





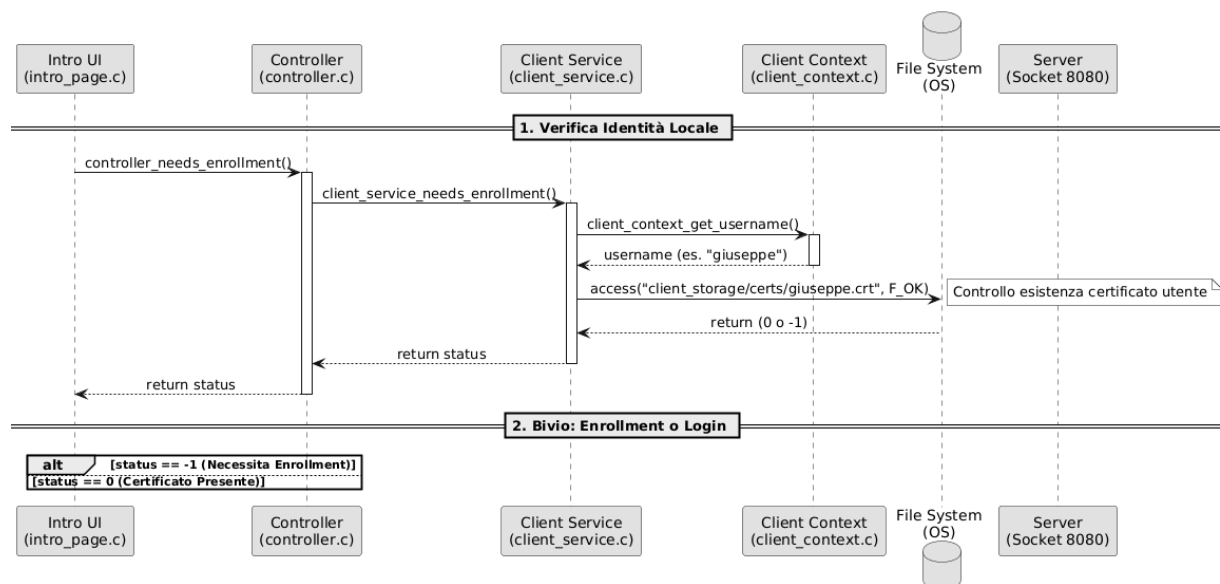
Inizio Enrollment e mTLS Connection



Sequence Diagram: Enrollment Start (Client)

Dopo aver configurato l'IP e importato la Root CA, il client si trova nella pagina di benvenuto.

Il diagramma seguente illustra cosa accade quando il client si **connette** al server per la prima volta senza possedere un certificato personale.



- La `needs_enrollment()` è semplicemente una funzione che andrà a verificare la presenza del **certificato** (`user.crt`), e nel caso predisporre l'utente alla fase successiva.

In assenza di un certificato dell'utente, allora viene predisposta la **procedura** di **enrollment**.

Focus Procedura di Enrollment

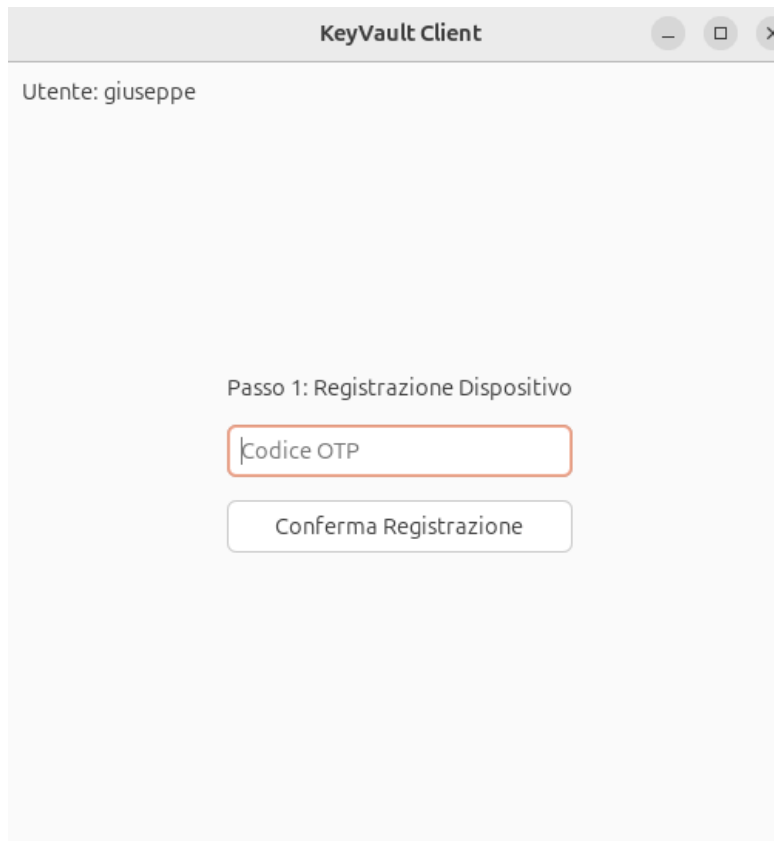
In questo scenario, il client non possiede ancora un certificato, ma deve interagire in modo sicuro con il server, e dunque con la **Root CA**, per ottenerne una.

La procedura di enrollment è progettata per essere sicura ma accessibile ai nuovi nodi. Essa si basa su tre pilastri fondamentali:

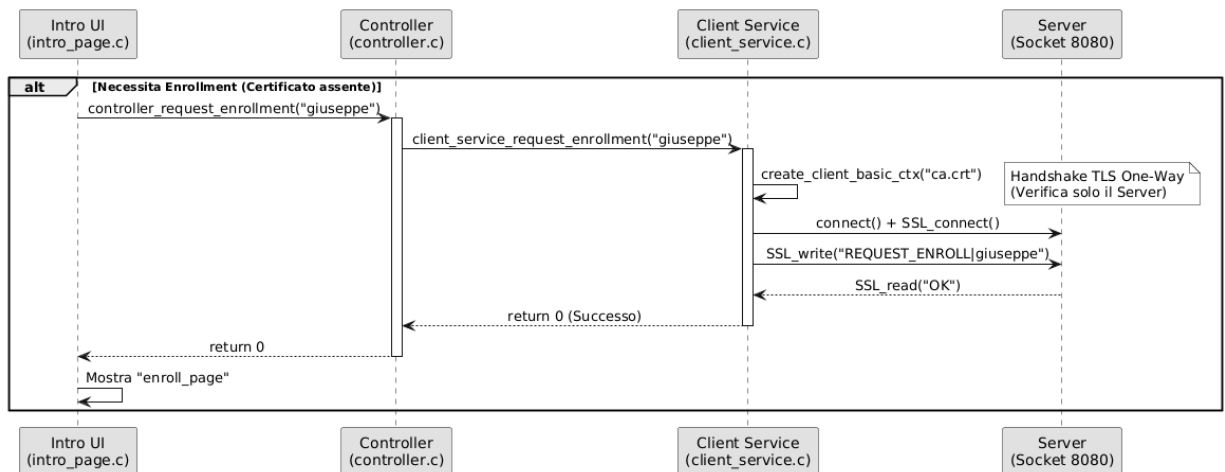
- Tunnel TLS Semplice (One-Way):** Il client non ha un certificato, quindi non può completare un Mutual TLS. Utilizza però la Root CA precedentemente importata per verificare che il server sia autentico, proteggendosi da attacchi Man-In-The-Middle durante la fase di registrazione.
- Codice OTP Out-Of-Band (OOB):** Per evitare che il server venga saturato da richieste di certificati automatizzate o non autorizzate, il processo richiede un codice OTP. Questo codice deve essere generato dal **server** e consegnato all'utente Out Of Band garantendo che solo l'utente legittimo possa proseguire.

Nota: Questo smistamento degli OTP OOB è una scelta coerente proprio per lo scenario aziendale o casalingo.

Il server, tramite la funzione `get_client_full_identity`, rileva l'assenza del certificato client. Invece di chiudere la connessione, restituisce uno stato di "Connessione Anonima" (`auth_status = 0`), istruendo il worker thread a gestire esclusivamente il protocollo di enrollment. (Vedi dopo)



The screenshot shows a window titled "KeyVault Client" with standard macOS window controls. Inside the window, the text "Utente: giuseppe" is displayed at the top. Below this, the heading "Passo 1: Registrazione Dispositivo" is centered. Under the heading, there is a text input field with the placeholder text "Codice OTP". Below the input field is a button labeled "Conferma Registrazione".

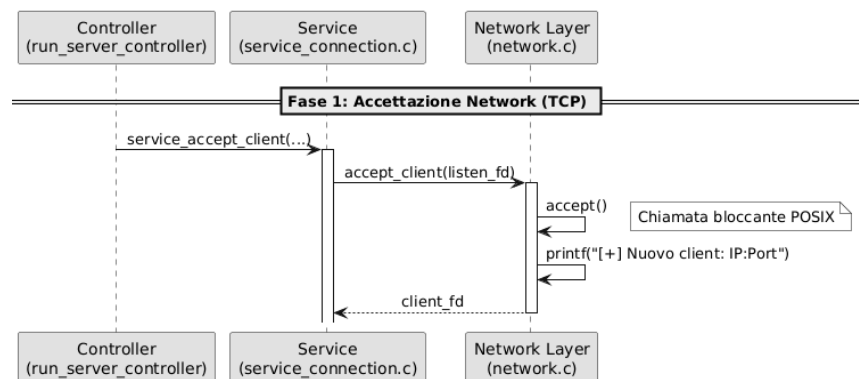


Una cosa interessante è proprio il fatto che in primis viene creato il **contesto base** (`create_client_basic_ctx`) e poi la creazione di una **connessione TCP** che viene **Upgradata** successivamente, rispettando proprio i **Layer**.

Sul canale TLS stabilito, viene dunque inviato una `Request_Enroll` e il Client attende poi l'**amministratore** per l'**OTP**. (Vedi Sotto)

Sequence Diagram: Enrollment Server

Il flusso inizia nel **Controller**, che invoca il servizio di accettazione. La prima operazione è delegata al **Network Layer**, il quale gestisce la "meccanica" del protocollo TCP:



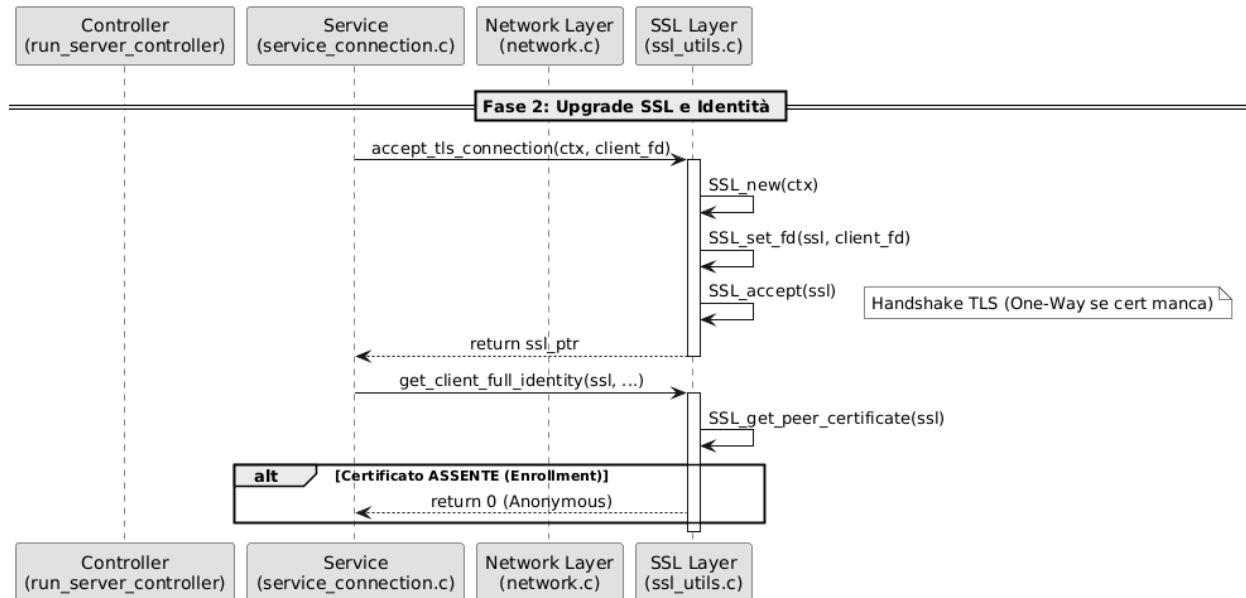
- `accept_client`: Esegue la system call `accept()`, bloccando il thread principale finché un client non completa il *three-way handshake*.

Una volta ottenuto il descrittore del socket (`client_fd`), il controllo passa allo strato **SSL**, che si occupa di upgradare la connessione:

- La funzione `accept_tls_connection` istanzia l'oggetto `SSL` per gestire la specifica connessione con il client, derivandolo dal **contesto globale** (`SSL_CTX`) precedentemente

configurato, e avvia la primitiva `SSL_accept()` per eseguire l'handshake.

Nota: Il server è configurato per non interrompere la connessione se il client non presenta un certificato `SSL_VERIFY_PEER`.



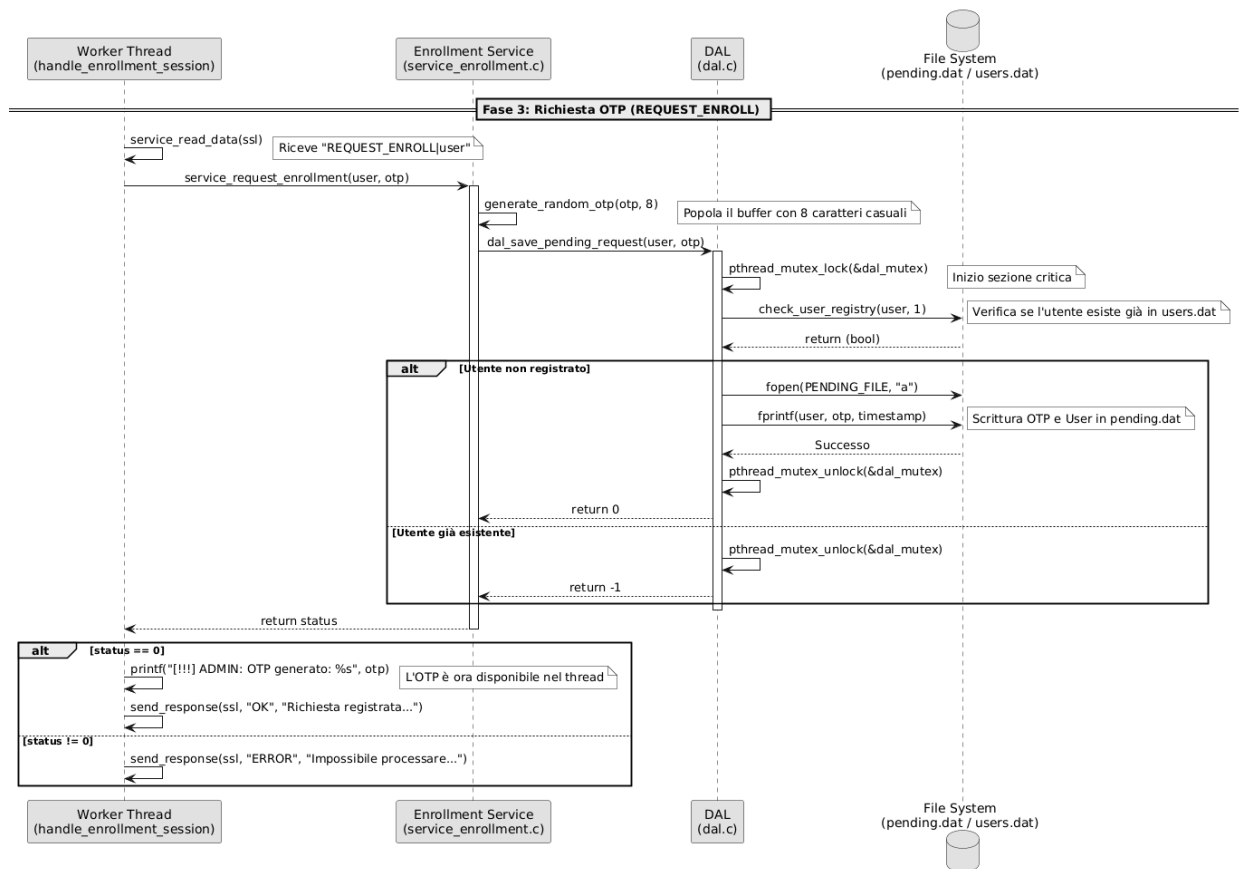
Il passaggio più critico avviene nella funzione `get_client_full_identity`, dove il server interroga il tunnel TLS per capire chi sia l'interlocutore:

- Con `SSL_get_peer_certificate` tentiamo di estrarre il **certificato**

Se il certificato è assente, la funzione restituisce **0 (Anonymous)**.

Questo segnale indica al sistema che il client è un nuovo nodo che richiede l'**attivazione** della **procedura** di **OTP** e la conseguente firma della **CSR** che arriverà dal Client.

In particolare, come detto alla fine dell'ultimo **Sequence Diagram**, il client invia sul **canale** TLS creato una `Request_Enrollment|user`.



Dunque il **Thread** designato opera il **Parsing** del messaggio e nota una **Request_Enrollment**, per cui genera un **OTP** associandolo all'**username** nel **pending.dat** in cui ci sono tutte le richieste pendenti di **firma** di **CSR**.

L'accesso ai file **users.dat** e **pending.dat** è condiviso tra i vari Thread, che potrebbero gestire contemporaneamente procedure di enrollment per più client, ecco perchè ho deciso di regolare l'**accesso in mutua esclusione**.

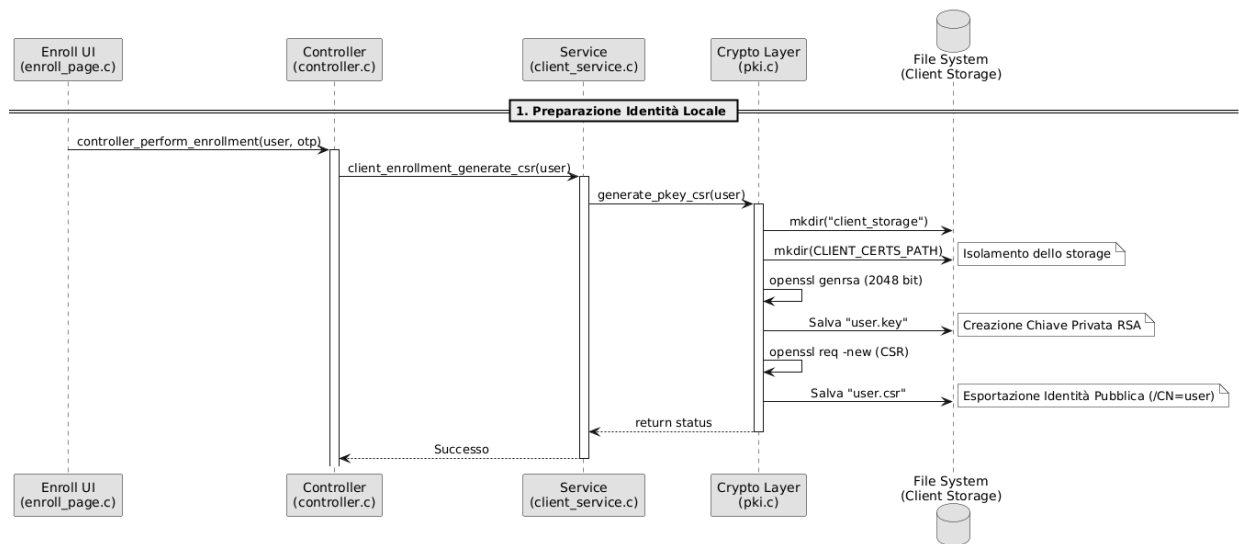
A questo punto la **connessione** col **client** viene **interrotta** in quanto l'admin deve far arrivare l'**OTP** al Client, per cui era inutile tenerla attiva.

Sequence Diagram: Invio OTP e Ricezione Certificato (Client)

Quando l'utente inserisce l'OTP ricevuto "Out-of-Band" e preme "Invia", il client avvia una sequenza di operazioni orchestrate dal Controller per fondamentalmente inviare insieme all'**OTP** anche una **CSR** e dunque il conseguente **materiale crittografico** necessario!

La funzione **controller_perform_enrollment** incarna perfettamente il ruolo del Controller dettando l'ordine del flusso di business:

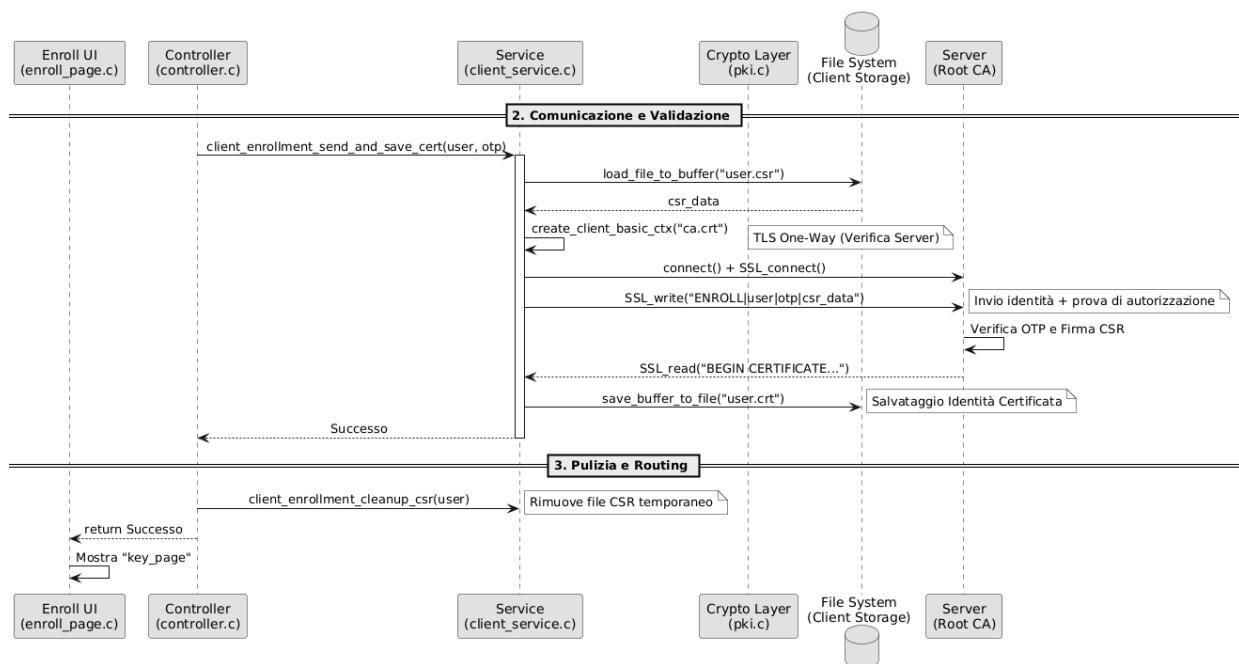
- **Fase di Generazione:** Ordina la creazione delle **chiavi** privata - pubblica e **CSR**.



Dunque in primis viene generata la **key privata** (RSA 2048 bit), da cui verrà poi estratta la chiave pubblica che verrà utilizzata per comporre la **CSR**, che poi verrà firmata con la **chiave privata** in modo che il **server** possa verificare che il client possenga la key privata corrispondente.

La **CSR** al momento viene salvata nel **file system** del **Client** in attesa di essere inviata.

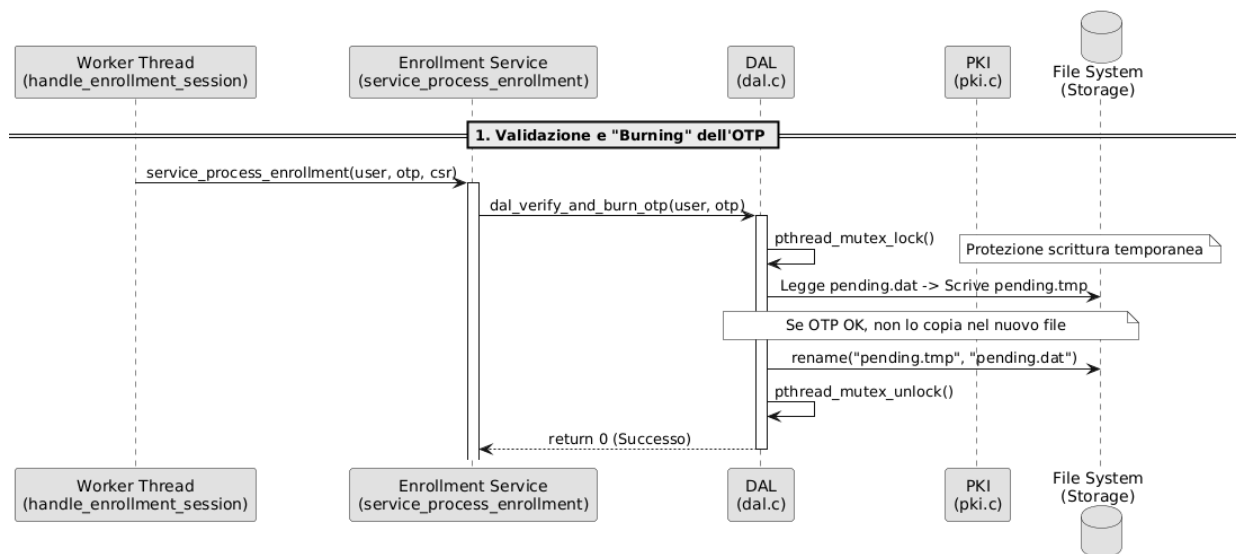
- **Fase di Scambio e Pulizia:** Ordina l'invio della richiesta e il salvataggio della risposta, assicurando la pulizia dei file temporanei (come il CSR) indipendentemente dall'esito, mantenendo lo storage del client ordinato e sicuro.



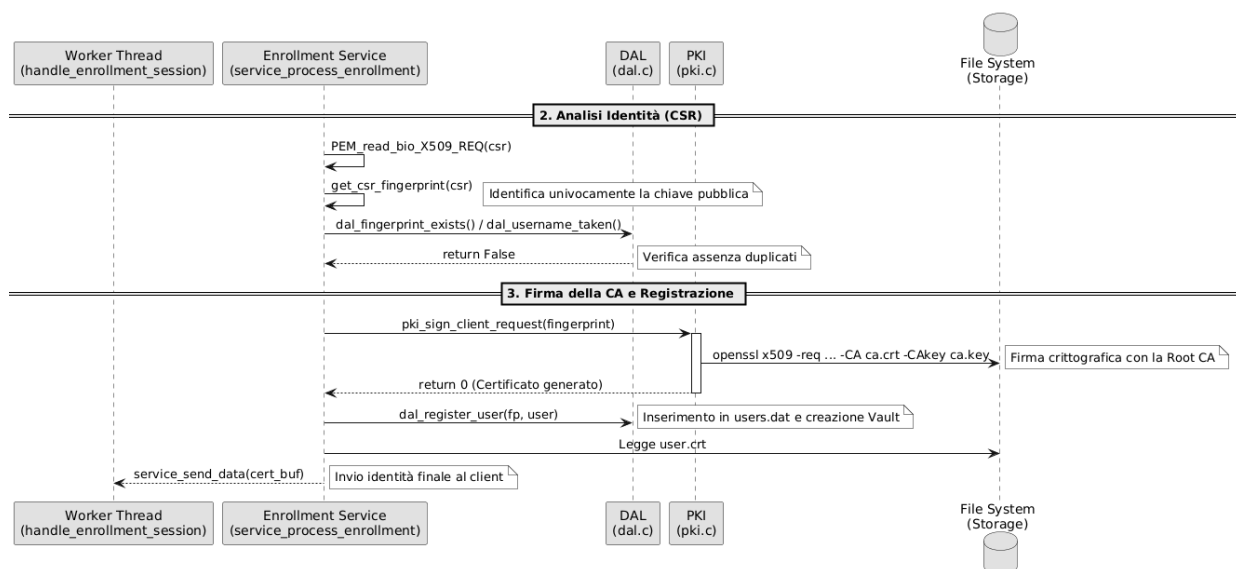
Viene caricato il file `user.csr` dal **filesystem**, per poi creare una nuova connessione TLS semplice su cui il **Client** invia il comando `ENROLL|user|otp|csr_data`.
Per cui adesso la palla passa al **Server** che deve verificare la validità dell'**OTP** e firmare eventualmente la **CSR**.

Sequence Diagram: Verifica OTP e Firma CSR

Il **thread** designato gestirà la comunicazione con la `handle_enrollment_session` che però adesso deve giusto verificare la validità dell'OTP e eventualmente firmare la **CSR**.



Nella prima fase, semplicemente c'è la fase di **validazione** e successive modifiche nei file `pending.dat`.



Nell'ultima fase, viene prelevato il **fingerprint** (hash chiave pubblica) dalla CSR ricevuta e salvata la **CSR** nel filesystem.

Il tutto però possibile solo se non ci sono eventuali **conflitti** tra **fingerprint** e **username** già registrati, per prevenire **Spoofing** dell'identità.

A questo punto avviene la vera e propria **firma** dalla **Root CA** del `fingerprint.csr` dell'utente e generato il `user.crt` che viene **inviato** al **client** (sempre sul canale TLS).

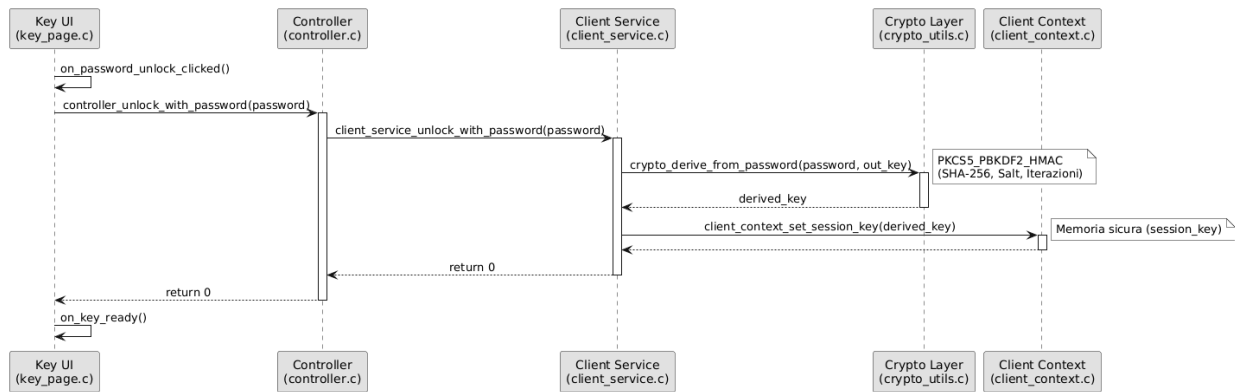
Termina così la fase di Enrollment.

Sequence Diagram: Scelta Key Encryption (Client)

A questo punto però si è chiusa la comunicazione TLS relativa all'enrollment, dunque questa fase avviene "offline": il Client ha ricevuto il proprio certificato che potrà usare per stabilire una connessione **mTLS**, ma prima deve passare per la fase di scelta della key di encryption-decryption da utilizzare per cifrare i dati, che arriveranno encrypted al Server per così **soddisfare** il requisito dell'**encryption at Rest**.



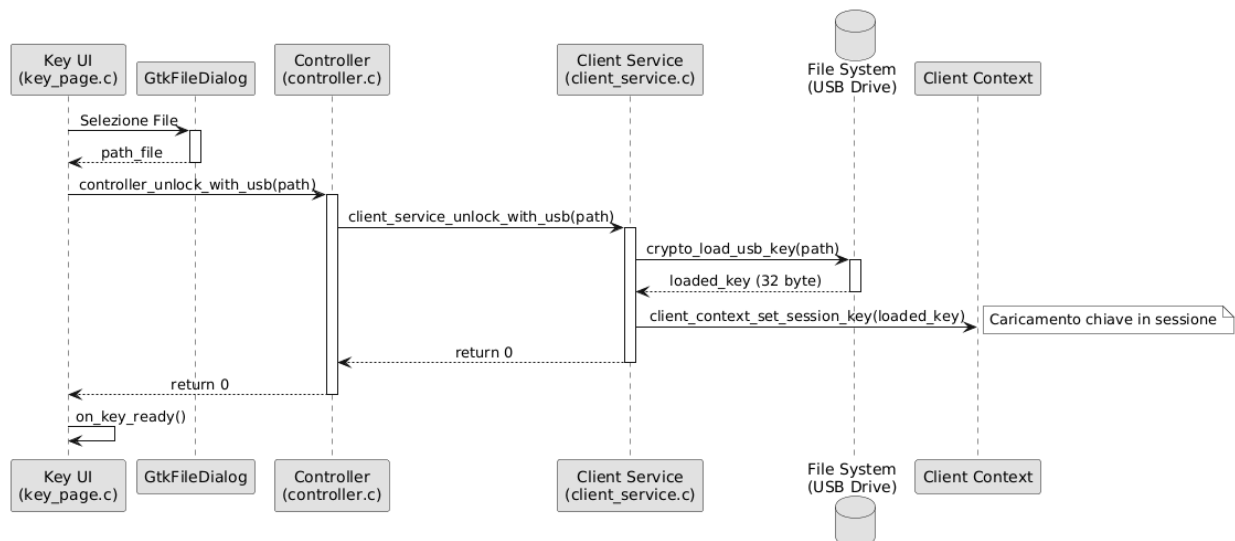
Sequence Diagram: Password Unlock



La funzione `crypto_derive_from_password` utilizza lo standard PKCS#5 con l'algoritmo HMAC-SHA256. L'uso di un **salt** statico e di un alto numero di iterazioni protegge la chiave da attacchi brute-force e rainbow tables, trasformando una password potenzialmente debole in una chiave AES a 256 bit.

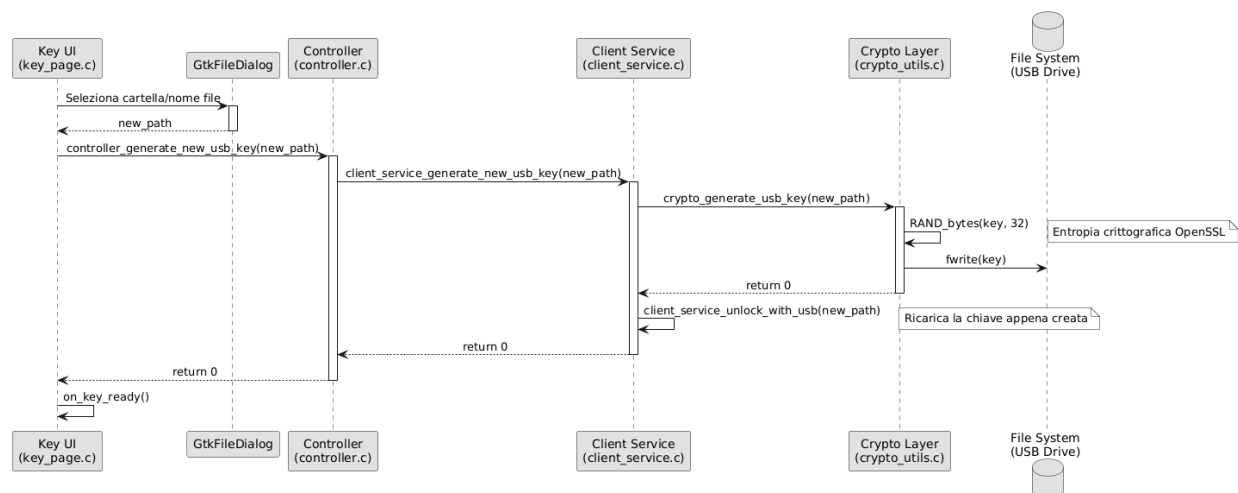
Sequence Diagram: Sblocco tramite File Chiave (USB)

L'utente può scegliere di caricare una chiave pre-esistente salvata su un supporto esterno (es. chiavetta USB). In questo caso, il file viene letto bit-a-bit.



Sequence Diagram Generazione di una Nuova Chiave

Se l'utente non possiede ancora una chiave, il sistema può generarne una nuova basata su entropia pura fornita dalla libreria `OpenSSL`.



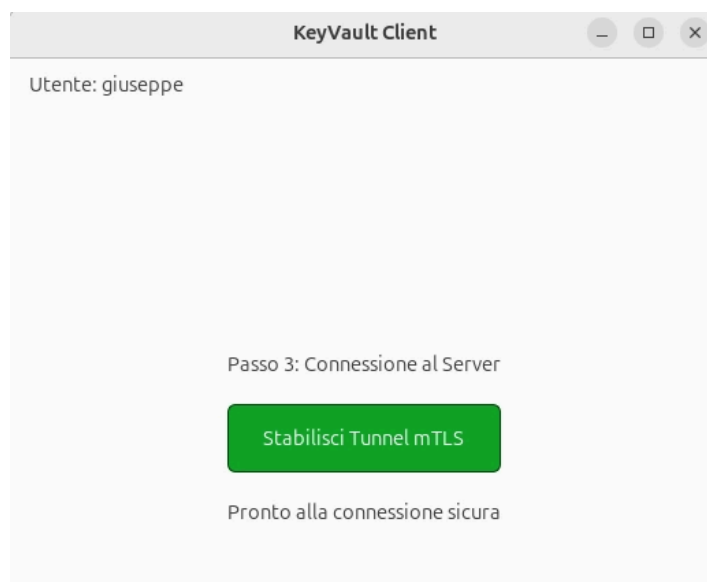
La sicurezza di questo metodo risiede nella funzione `RAND_bytes()` di OpenSSL. A differenza di un normale generatore di numeri pseudocasuali, questa funzione attinge a sorgenti di entropia del sistema operativo per generare una chiave statisticamente imprevedibile.

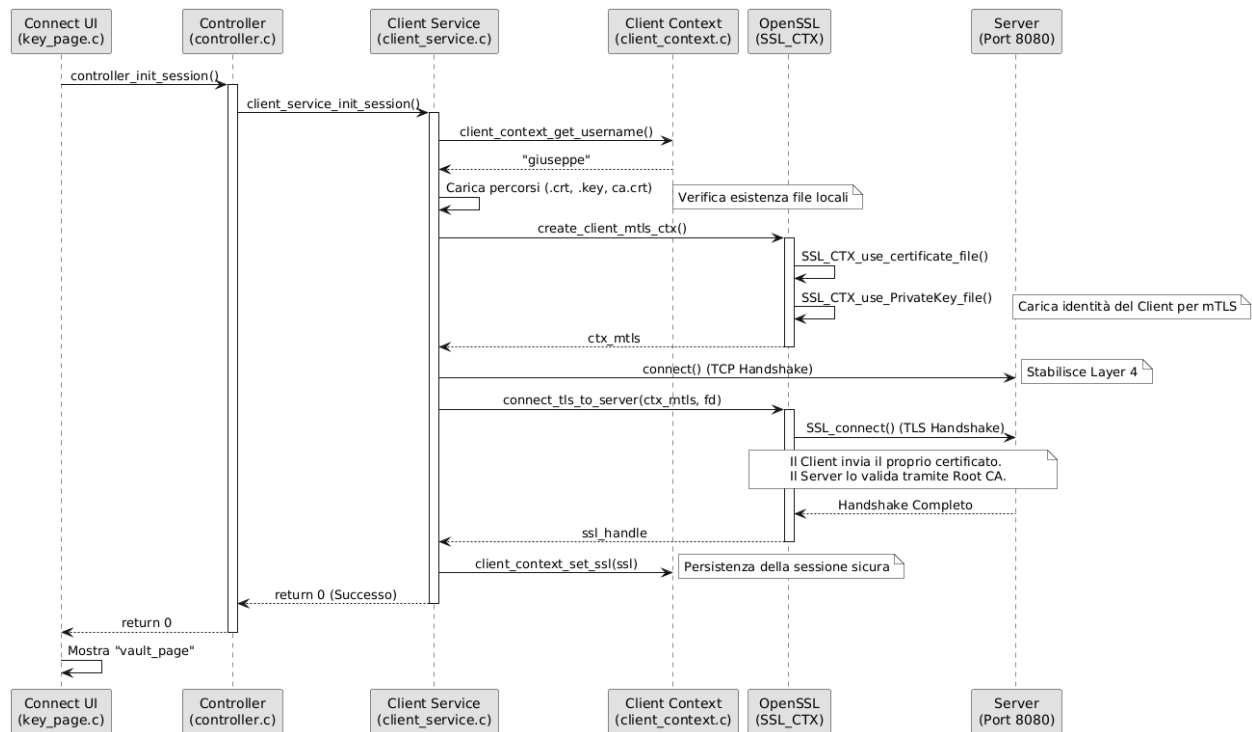
Nota: la **key** viene settata come **session key**, magari in futuro si potrebbe pensare di modificare questa funzionalità in quanto a causa di Memory Leak della **RAM** un attaccante potrebbe ottenerla.

Termina la fase di configurazione, sia per il client che per il server.

Sequence Diagram: Connessione mTLS (Client)

Quando l'utente clicca su "Connetti", il sistema avvia la procedura di inizializzazione della sessione sicura **mTLS**.



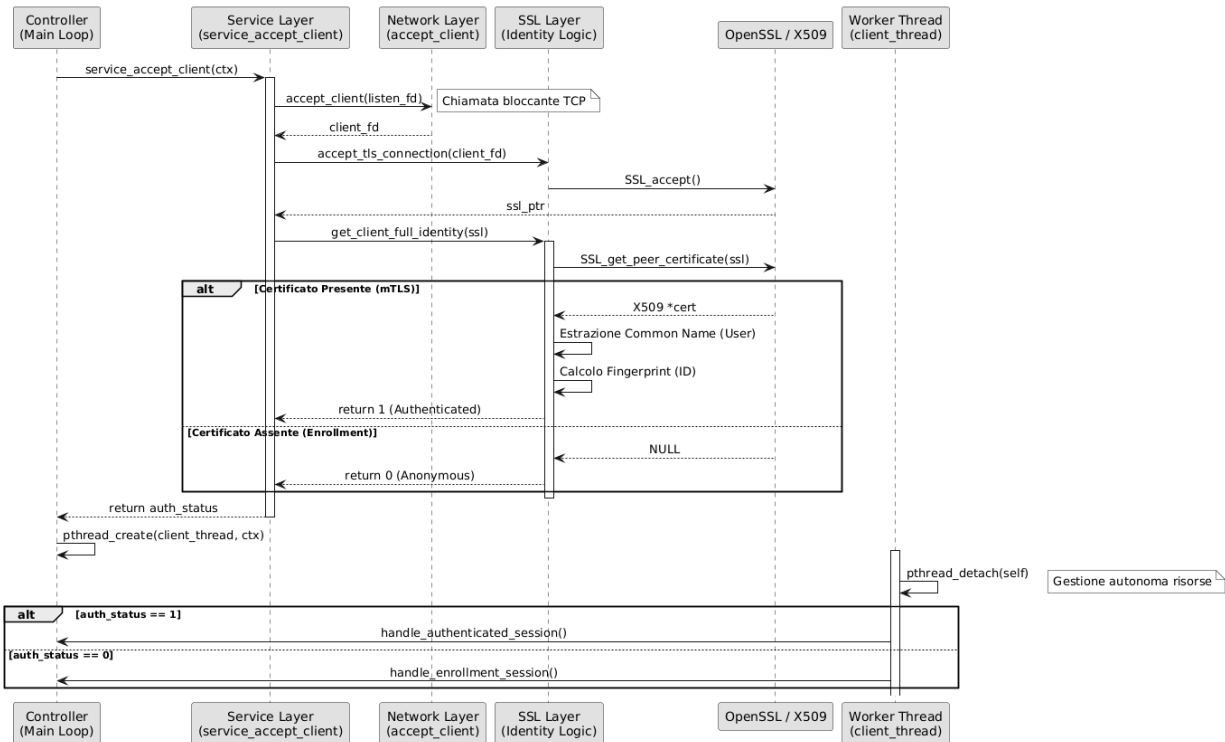


Il `basic_ctx` viene elevato a `mtls` caricando l'**identità** dell'utente: `user.crt` e `user.key`.
 Il Client invia dunque il proprio certificato **firmato** mediante la propria chiave privata, e attende la **validazione** del Server mediante la **Root CA** che dovrà **autenticare** il client.
 Terminato l'handshake, il Client mostrerà la **Vault Page** finale.

Sequence Diagram: Connessione mTLS (Server)

Il diagramma illustra come il server orchestra l'accettazione della connessione, l'estrazione dell'identità e la creazione del thread dedicato.

Questo diagramma è per lo più focalizzato sulla funzione `accept_client` in quanto adesso il **client** ha inviato anche il **certificato** durante l'handshake, come richiesto dal contesto configurato per il Server.



Una volta accettata la connessione, c'è la `get_client_full_identity` che adesso riesce a **estrarre il certificato**.

Fondamentalmente utilizzeremo il **Fingerprinting** e il **Common Name** per usi interni al Server (associare il vault all'utente).

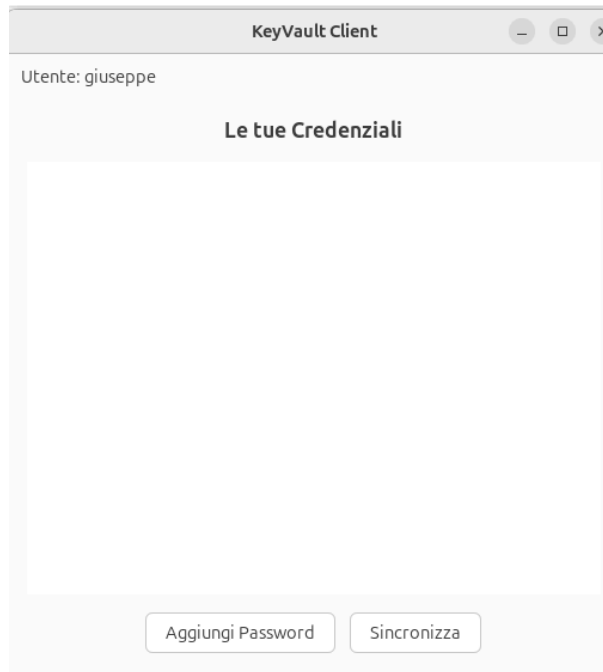
```
[+] Nuovo client connesso: 127.0.0.1:42156
[+] Connessione mTLS stabilita (User: giuseppe).
[*] Nuovo client accettato. Auth status: 1
[+] Controller: Sessione attiva per l'utente: giuseppe [ID: ecf7efbd...]
```

Nota: l'autenticazione del client dato mTLS, è intrinseca all'accettazione della connessione, dunque interamente gestito da OpenSSL.

Infatti inizialmente abbiamo caricato in OpenSSL il necessario per la **Root CA** e se il certificato non è valido a fallire sarà `SSL_accept`.

Gestione Credenziali nel Vault

Questa interfaccia è accessibile **esclusivamente** dopo che il Controller ha confermato lo stato di connessione mTLS attiva e la disponibilità della chiave crittografica (`crypto_ready`).

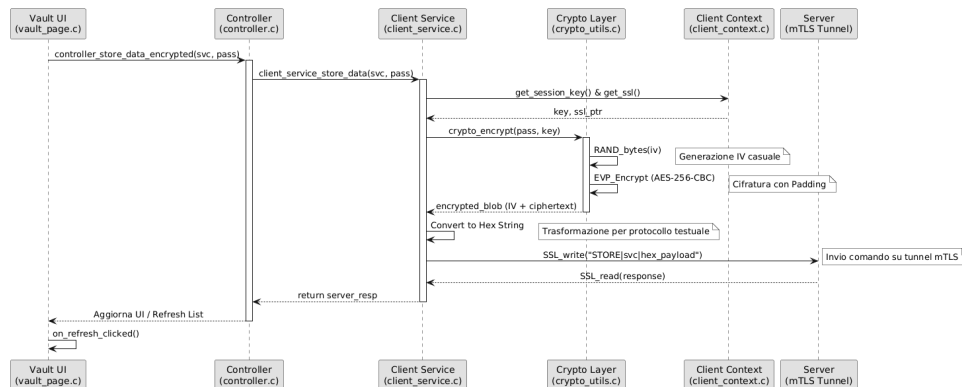
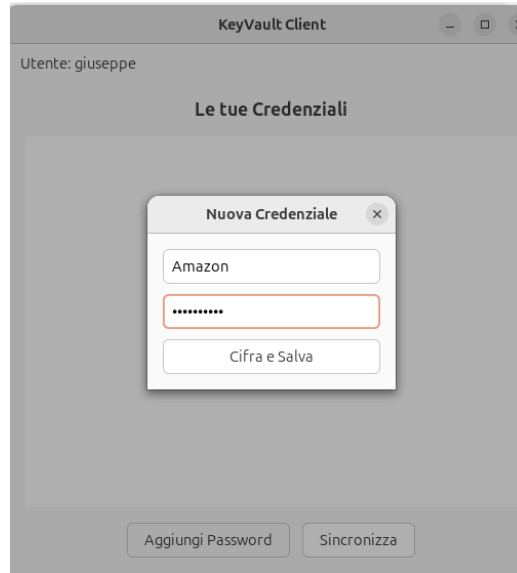


Archiviazione Sicura (STORE)

Il processo di salvataggio di una nuova password non invia mai dati in chiaro, nemmeno all'interno del tunnel cifrato.

Il sistema applica un **doppio strato di protezione**:

1. **Cifratura Locale**: I dati vengono cifrati con AES-256-CBC usando la chiave di sessione dell'utente, così che i dati che arrivano al server siano incomprensibili anche se avvenisse una compromissione di quest'ultimo.
2. **Cifratura di Trasporto (mTLS)**: Il payload già cifrato viene inviato attraverso il tunnel SSL.



Come funziona l'encryption?

La cifratura avviene all'interno del modulo `crypto_utils.c` tramite la funzione `crypto_encrypt`.

Il primo atto della funzione è la generazione di un **IV (Initialization Vector)** di 16 byte (128 bit).

```
RAND_bytes(iv, IV_LEN);
memcpy(out_buffer, iv, IV_LEN);
```

Sfruttando un **CBC** (Cipher Block Chaining), l'**IV** è fondamentale per garantire che stringhe di testo identiche producano **ciphertext** completamente diversi, avendo una **key privata statica**.

Quest'ultimo non è segreto, dato che è necessario per la fase di **decryption** (che farà sempre lo stesso client all'atto di una **fetch**).

Il blocco di dati finale infatti sarà composto da:

$[IV][Ciphertext]$

```
EVP_EncryptInit_ex(ctx, EVP_aes_256_cbc(), NULL, key, iv);
```

Il sistema utilizza l'interfaccia **EVP** (Envelope) di OpenSSL, un'astrazione di alto livello per utilizzare **implementazioni ottimizzate** indicando:

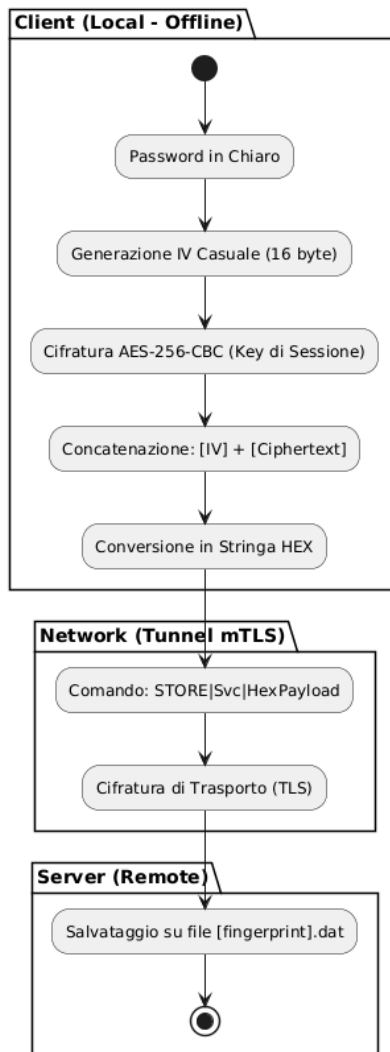
- **Algoritmo** `AES_256_CBC` che lavora su segmenti di 128 bit.
- **Chiave** `key` da 32 byte, derivata precedentemente dalla **master password** o caricata da USB.

A questo punto ottenuto il **blob binario** $[IV][Ciphertext]$, il Service deve renderlo compatibile con il **protocollo di comunicazione**, basato su stringhe.

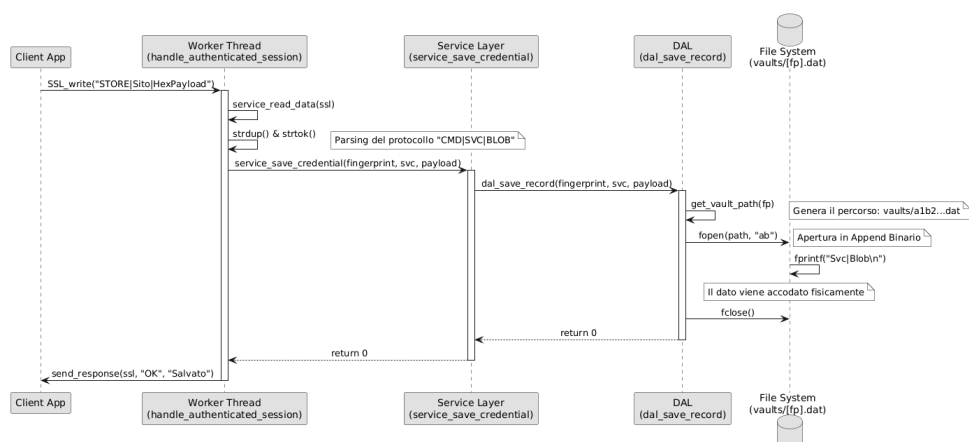
I dati cifrati infatti sono puramente binari, e possono contenere **byte** come `0x00` che il C interpreta come fine stringa o caratteri speciali che potrebbero corrompere il **protocollo** di comunicazione (`STORE|..|..|`).

Dunque ogni byte viene convertito in **2 caratteri esadecimali**.

Esempio: `0xFF` diventa `ff`.



Il Server riceve il comando **STORE**, ma non ha le chiavi per decifrare il payload. La sua unica responsabilità è garantire che quel "blob" esadecimale finisca nel file corretto, isolato da tutti gli altri utenti.



Il server associa ogni sessione a un `fingerprint` univoco estratto dal certificato mTLS. Poiché il nome del file fisico è il fingerprint stesso (`vaults/[fp].dat`), è matematicamente impossibile che due thread di utenti diversi scrivano sullo stesso file contemporaneamente.

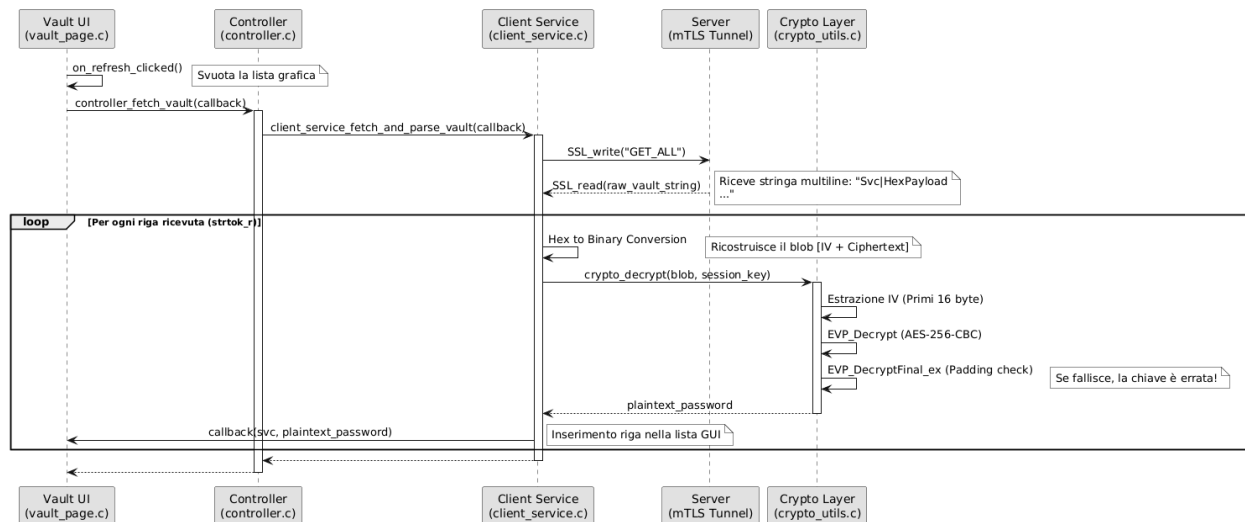
Questo design **lock-free** per utente permette al server di **scalare**: la velocità di scrittura di un utente non viene mai rallentata dalle attività degli altri utenti.

Poiché la decifratura avviene esclusivamente sul Client, il Server non ha mai visibilità sulle password memorizzate, riducendo drasticamente l'impatto di un'eventuale **compromissione** del **server** stesso: un attaccante otterrebbe solo file pieni di blob cifrati inutilizzabili senza le key degli utenti.

Flusso di Fetch (`GET_ALL`)

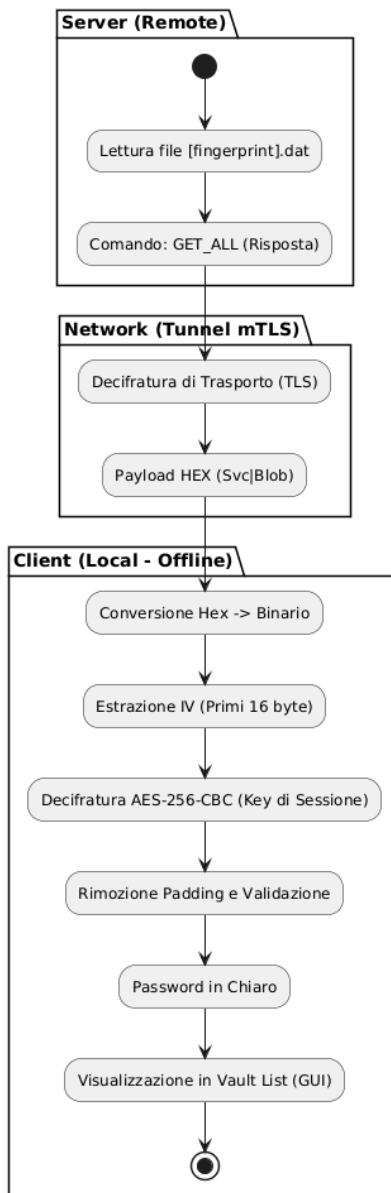
Con la fase di **Refresh/Fetch**, chiudiamo il ciclo di vita del dato. Questa operazione è il riflesso speculare della `STORE`: ora dobbiamo ottenere le password cifrate e riportarle in chiaro per l'utente.

Questa funzionalità viene invocata automaticamente sia all'accesso al Vault, sia dopo ogni salvataggio per garantire che la GUI sia sempre sincronizzata con il server.



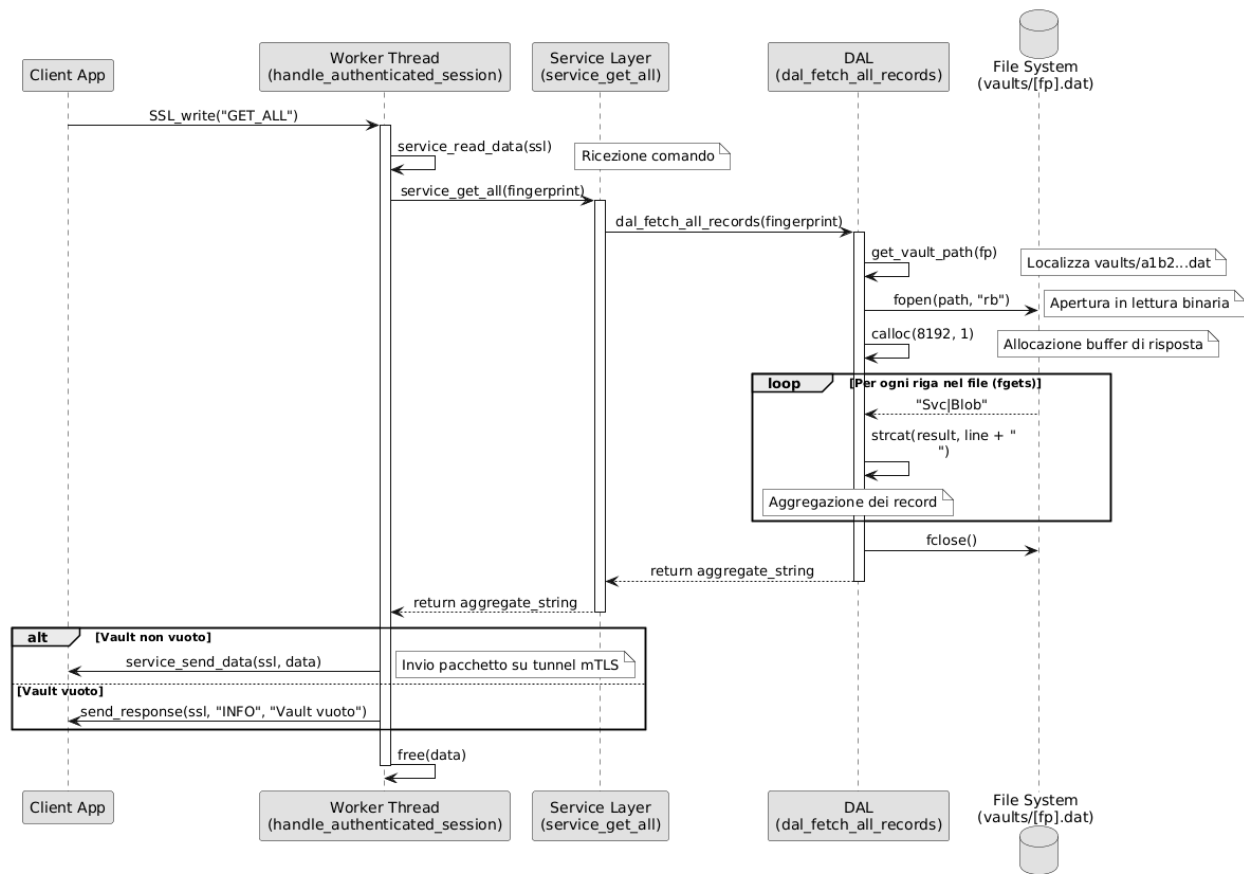
Come avviene la **decryption**?

Specularmente viene invocata la funzione `crypto_decrypt`, solo dopo però aver fatto **parsing** del messaggio che otteniamo dal Server.



Quando il client richiede la sincronizzazione, il **server** identifica il file fisico corretto, ne aggrega il contenuto e lo **spedisce** attraverso il tunnel mTLS già stabilito.

Il Data Access Layer (DAL) esegue una scansione lineare del file associato al fingerprint dell'utente:



Il server inviare l'intero Vault in un'unica risposta:

- Viene allocato un buffer (`result`) di circa 8KB. Questo spazio è sufficiente per contenere diverse decine di credenziali (considerando che ogni blob HEX AES-256 occupa circa 100-150 caratteri).
- La riga letta dal file (`servizio|hex_cifrato`) viene accodata così com'è.

Considerazioni Finali

Sicurezza della Memoria

Il Rischio del Persistent Keying

Nel modello di sessione tradizionale, una volta inserita la Master Password o caricata la chiave USB, il materiale crittografico rimane residente nella memoria RAM del processo per l'intera durata dell'esecuzione.

Questo design, sebbene comodo per l'utente, espone il sistema al rischio di **Memory Dump**: un attaccante con accesso al sistema potrebbe estrarre la chiave simmetrica analizzando i segmenti di memoria dell'applicazione.

Miglioramento: Architettura Zero-Persistence

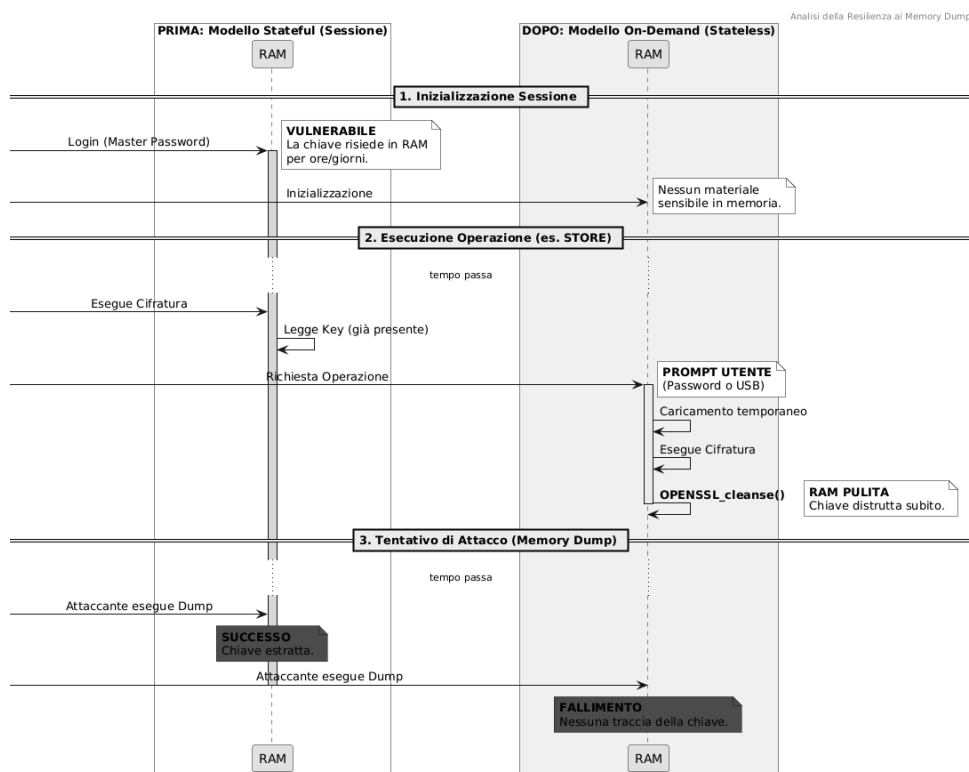
Per eliminare alla radice questa vulnerabilità, il sistema è stato evoluto verso un modello di **caricamento on-demand**. In questa configurazione, la chiave di sessione non viene mai memorizzata permanentemente nelle strutture dati del client (`ClientContext`).

L'accesso al materiale crittografico avviene con due modalità distinte in base al metodo scelto:

1. **Metodo Password**: Ad ogni operazione di cifratura (`STORE`) o decifratura (`GET_ALL`), l'utente deve inserire esplicitamente la Master Password.
2. **Metodo USB**: Il percorso del file chiave viene memorizzato nella fase di inizializzazione, ma il file viene aperto, letto in un buffer temporaneo, utilizzato e chiuso solo nel momento esatto dell'operazione.

Flusso Operativo

Il diagramma seguente mostra come la finestra di esposizione della chiave venga ridotta al solo tempo di esecuzione dell'algorithm AES.



Con questa implementazione, il vault raggiunge un livello di sicurezza **'Zero-Persistence'**. Il rischio di compromissione non è più legato alla durata della sessione dell'applicativo, ma viene ridotto a una frazione di secondo, rendendo il sistema resiliente anche agli attacchi più sofisticati basati sull'analisi della memoria volatile.

